A Comparative Study of Exact Solving Paradigms for the N-Queens Problem: SAT, CP, and ILP Approaches

Nguyen Cong Danh[a]

[a]Faculty of Information Technology, VNU University of Engineering and Technology, 144 Xuan Thuy, Cau Giay Ward, Hanoi, 100000, Vietnam

ARTICLE INFO

Keywords:
N-Queens
SAT solving
At-most-one encoding
Constraint programming
Integer linear programming
Feasibility

ABSTRACT

The N-Queens problem is a classical benchmark for constraint satisfaction and combinatorial optimization. Its SAT formulation exposes a central representational challenge: controlling the combinatorial growth of pairwise conflict clauses when translating at-most-one (AMO) constraints into conjunctive normal form (CNF). This paper presents a comprehensive empirical study of SAT, constraint programming (CP), and integer linear programming (ILP) across 18 board sizes, from $n = 4$ to $n = 1331$, using 14 solver configurations and 252 benchmark runs. Five AMO encodings—Pairwise (Binomial), Binary, Sequential Counter, Commander, and recursive 2-Product—are evaluated with the conflict-driven clause-learning solvers Glucose 3.0 and CaDiCaL 1.9.5. These ten SAT configurations are compared directly with OR-Tools CP-SAT, IBM CPLEX CP Optimizer, Gurobi, and CPLEX MIP. The evaluation measures CNF size, model construction time, solver-call time, and peak sampled memory usage. Commander with Glucose 3.0 is the only SAT configuration to solve all 18 sizes, reaching $n = 1331$ in 85.08 seconds. IBM CPLEX CP Optimizer exhibits the strongest overall scaling, solving every tested size in under 0.73 seconds and $n = 1331$ in 0.40 seconds; these runtimes include model construction and the solver call. The results reveal a "clause-count paradox": smaller CNF formulations do not necessarily yield faster solutions. This finding motivates evaluating propagation structure, including unit-propagation depth, alongside clause counts and solver-specific search behavior. Overall, the study demonstrates that compact encodings and computational efficiency are distinct criteria, making the interaction between constraint representation and solver choice central to empirical performance.

1. Introduction

1.1. Context and Problem Scope

The N-Queens problem is a classical benchmark for constraint satisfaction: place n queens on an $n \times n$ chessboard such that no two queens share a row, column, or diagonal. Its concise specification supports several distinct mathematical formulations, making it a useful benchmark for evaluating constraint propagation, search behavior, model construction, and memory consumption in exact solvers. The underlying task of selecting mutually compatible assignments also reflects constraint patterns arising in resource scheduling, VLSI layout, and telecommunications routing. Here, N-Queens provides a controlled setting for comparing representations of such conflicts.

This study addresses the ordinary N-Queens feasibility problem, in which the board is initially empty and the objective is to obtain one valid placement. This problem admits constructive solutions and must be distinguished from N-Queens Completion, where a prescribed partial placement must be extended to a complete solution. The latter problem is NP-complete [1]. Our focus is the computational behavior of general-purpose exact solvers under alternative formulations of ordinary N-Queens.

1.2. Modeling Paradigms and Encoding Trade-offs

We investigate three modeling paradigms: constraint programming (CP), integer linear programming (ILP), and Boolean satisfiability (SAT). The CP formulation represents a placement by an integer vector $Q = (q_0, \ldots, q_{n-1})$, where q_i identifies the column occupied by the queen in row i. Three global AllDifferent constraints enforce distinct columns and distinct values of $q_i + i$ and $q_i - i$, respectively.

The ILP formulation introduces an $n \times n$ matrix of binary variables. Row and column equalities require exactly one queen on each row and column, while diagonal inequalities permit at most one queen on each diagonal. The SAT formulation uses the same cell-based representation and expresses these requirements in conjunctive normal form (CNF). Rows and columns receive both at-least-one (ALO) and at-most-one (AMO) constraints; diagonals receive AMO constraints.

A central design choice in SAT is the encoding of AMO constraints. For a group of k variables, Pairwise, also called Binomial, generates $\binom{k}{2}$ clauses without auxiliary variables. Applied across the board, this yields $\Theta(n^3)$ clauses. Auxiliary-variable encodings offer different space trade-offs: Binary uses $\Theta(k \log k)$ clauses per AMO constraint and $\Theta(n^2 \log n)$ clauses for the complete board, whereas Sequential Counter, Commander, and recursive 2-Product use $\Theta(k)$ clauses per constraint and $\Theta(n^2)$ clauses overall. These reductions change both the formula size and the structure through which implications are propagated.

1.3. Research Problem and the Clause-Count Paradox

Does a smaller CNF formulation necessarily produce a faster solution on a conflict-driven clause-learning (CDCL) SAT solver? Our experiments expose a clause-count paradox: reductions in clause count do not consistently translate into shorter runtimes. In particular, the compact recursive 2-Product encoding does not achieve the strongest SAT performance, while Commander with Glucose 3.0 is the only SAT configuration to solve all 18 tested board sizes.

This observation makes encoding–solver interaction a central evaluation target. Encodings differ in their auxiliary-variable organization, implication paths, and unit-propagation depth. CDCL implementations also differ in preprocessing, inprocessing, branching heuristics, such as VSIDS and its variants, and phase-selection policies, including rephasing where implemented. The empirical question is therefore how alternative CNF structures perform with each backend, rather than whether clause count alone predicts efficiency. Our comparison measures these interactions through model size, construction cost, solver-call time, and memory usage.

1.4. Research Contributions

The contributions of this study are fourfold:

- **Comprehensive empirical benchmark.** We conduct 252 benchmark runs, each executed in a separate process, covering 14 solver configurations and 18 board sizes:

  $$n \in \{4, 8, 9, 16, 25, 27, 36, 49, 64,\\ 81, 100, 125, 216, 343, 512, 729, 1000, 1331\}.$$

  These selected square and cube sizes span approximately five orders of magnitude in the number of board cells. Each run is subject to an externally monitored 100-second process budget and a memory threshold of 14 GiB (14,336 MiB).

- **Comparative SAT AMO encoding analysis.** We evaluate Pairwise, Binary, Sequential Counter, Commander, and recursive 2-Product with Glucose 3.0 and CaDiCaL 1.9.5, comparing variable counts, clause counts, runtimes, memory consumption, and the number of successfully solved sizes.

- **Cross-paradigm performance mapping.** We compare the ten SAT configurations directly with IBM CPLEX CP Optimizer and OR-Tools CP-SAT, as well as the binary ILP formulations solved by Gurobi and CPLEX MIP. CP Optimizer and Commander with Glucose solve all 18 sizes; Gurobi and CPLEX MIP solve 17 and 16 sizes, respectively.

- **System-level performance analysis.** We distinguish the measured model-construction interval (`Build_Time`) from the elapsed solver-call interval (`Solve_Time`) and analyze their sum (`Total_Time`). We additionally measure peak sampled process-tree resident set size (RSS), expressed in MiB. This separation identifies the contribution of model construction to recorded runtime while accounting for the memory footprint of the executing processes.

1.5. Paper Organization

Section 2 presents the problem definition and the CP and ILP formulations. Section 3 specifies the SAT model, the five AMO encodings, and their space requirements. Section 4 describes the experimental setup and compares model sizes, runtimes, memory consumption, and solving coverage. Section 5 summarizes the principal findings.

2. Background and Baseline Formulations

2.1. Problem Definition and Mathematical Notation

Let $n \geq 1$ and let $I_n = \{0, \dots, n-1\}$ denote the zero-based row and column indices of an $n \times n$ chessboard. The ordinary N-Queens feasibility problem seeks a set $P \subseteq I_n \times I_n$ of exactly n occupied cells such that no two distinct queens attack each other. Two queens at (i, j) and (i', j') conflict precisely when at least one of the following conditions holds:

$$\begin{aligned} i &= i', && \text{same row,} \\ j &= j', && \text{same column,} \\ i - j &= i' - j', && \text{same main diagonal,} \\ i + j &= i' + j', && \text{same anti-diagonal.} \end{aligned} \tag{1}$$

Main diagonals are indexed by $d \in \{1-n, \dots, n-1\}$, where $i-j = d$; anti-diagonals are indexed by $s \in \{0, \dots, 2n-2\}$, where $i + j = s$. We use d for the main-diagonal index to distinguish it from the CP decision variables q_i introduced below.

As illustrated in Figure 1(a), a valid placement requires that no queen lie on another queen's line of attack, whereas Figure 1(b) demonstrates an invalid placement where multiple queens threaten each other along shared rows and diagonals. Geometric intersections between attack lines at unoccupied cells are permitted: a conflict occurs when one queen occupies a cell attacked by another.

Since the placement contains n queens and no two share a row or column, every row and column contains exactly one queen. Diagonals, in contrast, may be empty and therefore require only at-most-one constraints. These conditions define a feasibility problem without a nonconstant objective function.

2.2. Related Work and Baseline Formulations

N-Queens has long served as a test problem for systematic backtracking and constraint propagation, predating the widespread adoption of modern CDCL SAT solvers. Its structure connects naturally to finite-domain constraint programming through permutation variables and to integer programming through binary assignment variables. These formulations provide complementary baselines: CP expresses non-attacking conditions through global constraints, while ILP represents the same conditions through linear equalities and inequalities.

2.2.1. Constraint Programming Model

Constraint programming represents decisions using finite integer domains and combines them through constraints that support domain propagation. For N-Queens, define

$$Q = (q_0, \dots, q_{n-1}), \qquad q_i \in I_n \quad \forall i \in I_n, \tag{2}$$

where q_i is the column occupied by the queen in row i.

- **Row constraints.** Each row has exactly one associated variable q_i, whose assigned value selects one column. Exactly one queen per row is therefore enforced by the representation itself, using n integer variables instead of n^2 binary cell variables.
- **Column constraints.** Distinct rows must select distinct columns:

$$\mathrm{AllDifferent}(q_0, q_1, \dots, q_{n-1}). \tag{3}$$

- **Diagonal constraints.** The two diagonal families are excluded through index offsets:

$$\begin{aligned} &\mathrm{AllDifferent}\big(q_0 + 0, q_1 + 1, \dots, q_{n-1} + (n-1)\big), \\ &\mathrm{AllDifferent}\big(q_0 - 0, q_1 - 1, \dots, q_{n-1} - (n-1)\big). \end{aligned} \tag{4}$$

The first constraint separates anti-diagonal indices $i + q_i$. The second separates $q_i - i$, equivalently the main-diagonal indices $i - q_i$.

Together, Equations (3) and (4) impose the three global AllDifferent constraints used by the IBM CPLEX CP Optimizer and OR-Tools CP-SAT baselines.

A key theoretical advantage of the global AllDifferent constraint is the availability of matching-based propagation. Régin's filtering algorithm represents variables and their possible values as a bipartite graph and enforces generalized arc consistency (GAC): every retained variable–value assignment belongs to a satisfying assignment of that individual global constraint. The CP formulation contains n integer decision variables, each initially having n possible values; consequently, an explicit variable–value graph can contain n^2 edges. This distinguishes the linear number of decision variables from the size of their domain representation.

2.2.2. Integer Linear Programming Model

Integer linear programming expresses the placement conditions as linear equalities and inequalities over binary variables. Let $X = (X_{i,j})_{i,j \in I_n}$, where

$$X_{i,j} \in \{0,1\}, \qquad X_{i,j} = 1 \iff \text{cell } (i,j) \text{ contains a queen.} \tag{5}$$

The complete formulation consists of four constraint families.

- **Exactly one queen per row:**

$$\sum_{j=0}^{n-1} X_{i,j} = 1, \qquad \forall i \in I_n. \tag{6}$$

- **Exactly one queen per column:**

$$\sum_{i=0}^{n-1} X_{i,j} = 1, \qquad \forall j \in I_n. \tag{7}$$

- **At most one queen per main diagonal:**

$$\sum_{\substack{0 \le i,j < n \\ i-j=d}} X_{i,j} \le 1, \qquad \forall d \in \{1-n, \dots, n-1\}. \tag{8}$$

- **At most one queen per anti-diagonal:**

$$\sum_{\substack{0 \le i,j < n \\ i+j=s}} X_{i,j} \le 1, \qquad \forall s \in \{0, \dots, 2n-2\}. \tag{9}$$

This formulation is used by both Gurobi and CPLEX MIP. Before presolve, it contains exactly n^2 binary variables and

$$n + n + (2n-1) + (2n-1) = 6n - 2 \tag{10}$$

linear constraints. This count retains every diagonal, including singleton corner diagonals whose inequalities are already implied by the binary variable bounds. Variable bounds and integrality declarations are separate from these $6n - 2$ linear constraints.

Each variable $X_{i,j}$ appears with coefficient 1 in exactly four constraints: its row, column, main diagonal, and anti-diagonal. The resulting constraint matrix therefore contains exactly

$$\text{nnz}(A) = 4n^2 \tag{11}$$

nonzero entries before presolve.

The CP and ILP formulations describe the same placements. Given a CP solution Q, set $X_{i,q_i} = 1$ and all other entries to zero. Conversely, the unique nonzero entry in each row of an ILP solution determines q_i. This correspondence establishes an equivalent feasibility target for the baseline comparison.

3. Methodology

3.1. Common board formula and variable indexing

The SAT variable $B_{i,j}$ has identifier

$$\mathrm{id}(B_{i,j}) = in + j + 1, \qquad 0 \le i, j < n. \tag{12}$$

Thus board coordinates are zero-based and SAT identifiers are positive and one-based. Auxiliary identifiers are allocated consecutively from $n^2 + 1$. Every invocation of an encoding, including recursive invocations, allocates fresh auxiliaries. Auxiliaries belonging to different board lines are never shared. The SAT and ILP variables represent the same cell event: $B_{i,j}$ is true exactly when $x_{i,j} = 1$.

Use the following ordered lists; diagonal entries are ordered by increasing i:

$$\begin{aligned} R_i &= (B_{i,0}, \dots, B_{i,n-1}), & C_j &= (B_{0,j}, \dots, B_{n-1,j}), \\ D_d &= (B_{i,i-d} : 0 \le i < n,\ 0 \le i - d < n), \\ A_s &= (B_{i,s-i} : 0 \le i < n,\ 0 \le s - i < n). \end{aligned} \tag{13}$$

For a list $X = (X_0, \dots, X_{k-1})$, let $E(X)$ denote one of the five AMO encodings and define $\mathrm{ALO}(X) = \bigvee_{t=0}^{k-1} X_t$. The complete formula is

$$\begin{aligned} F_E(n) = &\bigwedge_{i=0}^{n-1} \big(\mathrm{ALO}(R_i) \wedge E(R_i)\big) \\ &\wedge \bigwedge_{j=0}^{n-1} \big(\mathrm{ALO}(C_j) \wedge E(C_j)\big) \\ &\wedge \bigwedge_{d=1-n}^{n-1} E(D_d) \wedge \bigwedge_{s=0}^{2n-2} E(A_s). \end{aligned} \tag{14}$$

Only rows and columns receive ALO clauses. No diagonal ALO is added. For $k \le 1$, $E(X) = \top$ and no auxiliary or clause is generated. This convention matches the early return in `amo`. The row and column ALO clauses are both retained even though some constraints are logically redundant. At $n = 1$, the implementation emits two identical positive unit clauses and uses one board variable.

Correctness of an auxiliary encoding means

$$\mathrm{AMO}(X) \iff \exists Y\ E(X, Y), \tag{15}$$

where Y is its fresh auxiliary set. It does not require every auxiliary to have a uniquely determined value. Counts below refer to emitted clauses before solver preprocessing, including any redundant clauses.

3.2. Binomial (Pairwise) Encoding

The Binomial encoding, also called Pairwise encoding, enforces an at-most-one constraint by explicitly prohibiting every pair of variables from being simultaneously true. Each forbidden pair is represented by a binary clause containing two negative literals.

3.2.1. Encoding Variables

- **Decision variables.** For an $n \times n$ board, introduce

  $$B_{i,j} \in \{0, 1\}, \qquad 0 \le i, j < n, \tag{16}$$

  where $B_{i,j}$ is true if and only if a queen occupies cell (i, j). Both coordinates use zero-based indexing. For example, on a 4×4 board, $B_{2,3}$ being true indicates a queen in row 2, column 3.

- **Auxiliary variables.** Pairwise encoding places its clauses directly on the decision variables $B_{i,j}$. It introduces exactly **zero auxiliary variables**. Consequently, the 4×4 instance contains precisely 16 CNF variables.

3.2.2. Encoding Constraints

(a) Exactly One Queen Per Row. Each row must contain exactly one queen. This requirement combines an at-most-one (AMO) constraint with an at-least-one (ALO) constraint.

At most one queen per row. Every pair of distinct cells in the same row is prohibited from simultaneously containing queens:

$$F_{\text{row}}^{\text{AMO}} = \bigwedge_{i=0}^{n-1} \bigwedge_{0 \le j < j' < n} \left(\neg B_{i,j} \vee \neg B_{i,j'} \right). \tag{17}$$

For $n = 4$, the constraint on row 0 expands into the following six binary clauses:

$$\begin{aligned} F_{\text{row},0}^{\text{AMO}} = {} & (\neg B_{0,0} \vee \neg B_{0,1}) \\ & \wedge (\neg B_{0,0} \vee \neg B_{0,2}) \\ & \wedge (\neg B_{0,0} \vee \neg B_{0,3}) \\ & \wedge (\neg B_{0,1} \vee \neg B_{0,2}) \\ & \wedge (\neg B_{0,1} \vee \neg B_{0,3}) \\ & \wedge (\neg B_{0,2} \vee \neg B_{0,3}). \end{aligned} \tag{18}$$

At least one queen per row. A single positive clause requires a queen in each row:

$$F_{\text{row}}^{\text{ALO}} = \bigwedge_{i=0}^{n-1} \left(\bigvee_{j=0}^{n-1} B_{i,j} \right). \tag{19}$$

For $n = 4$, the four row ALO clauses are

$$\begin{aligned} F_{\text{row}}^{\text{ALO}} = {} & (B_{0,0} \vee B_{0,1} \vee B_{0,2} \vee B_{0,3}) \\ & \wedge (B_{1,0} \vee B_{1,1} \vee B_{1,2} \vee B_{1,3}) \\ & \wedge (B_{2,0} \vee B_{2,1} \vee B_{2,2} \vee B_{2,3}) \\ & \wedge (B_{3,0} \vee B_{3,1} \vee B_{3,2} \vee B_{3,3}). \end{aligned} \tag{20}$$

Together, the row AMO and ALO clauses permit exactly one true variable in each row.

(b) Exactly One Queen Per Column. The same construction is applied to columns.

At most one queen per column. For each column, every pair of distinct rows contributes one binary exclusion clause:

$$F_{\text{col}}^{\text{AMO}} = \bigwedge_{j=0}^{n-1} \bigwedge_{0 \le i < i' < n} \left(\neg B_{i,j} \vee \neg B_{i',j} \right). \tag{21}$$

For $n = 4$, column 0 requires the following six clauses:

$$\begin{aligned} F_{\text{col},0}^{\text{AMO}} = {} & (\neg B_{0,0} \vee \neg B_{1,0}) \\ & \wedge (\neg B_{0,0} \vee \neg B_{2,0}) \\ & \wedge (\neg B_{0,0} \vee \neg B_{3,0}) \\ & \wedge (\neg B_{1,0} \vee \neg B_{2,0}) \\ & \wedge (\neg B_{1,0} \vee \neg B_{3,0}) \\ & \wedge (\neg B_{2,0} \vee \neg B_{3,0}). \end{aligned} \tag{22}$$

At least one queen per column. Each column receives one positive clause:

$$F_{\text{col}}^{\text{ALO}} = \bigwedge_{j=0}^{n-1} \left(\bigvee_{i=0}^{n-1} B_{i,j} \right). \tag{23}$$

For $n = 4$, this gives

$$\begin{aligned} F_{\text{col}}^{\text{ALO}} = {} & (B_{0,0} \vee B_{1,0} \vee B_{2,0} \vee B_{3,0}) \\ & \wedge (B_{0,1} \vee B_{1,1} \vee B_{2,1} \vee B_{3,1}) \\ & \wedge (B_{0,2} \vee B_{1,2} \vee B_{2,2} \vee B_{3,2}) \\ & \wedge (B_{0,3} \vee B_{1,3} \vee B_{2,3} \vee B_{3,3}). \end{aligned} \tag{24}$$

The formulation retains both row and column ALO clauses, matching the implemented construction.

(c) At Most One Queen Per Diagonal. Every diagonal may contain at most one queen. No diagonal ALO clause is imposed because a diagonal may remain empty. Only diagonals containing at least two cells contribute pairwise clauses.

Main diagonals and their parallels. Cells on the same main diagonal satisfy $i - j = q$. The non-singleton diagonals have $q \in \{2 - n, \dots, n - 2\}$, giving

$$F_{\text{main}}^{\text{AMO}} = \bigwedge_{q=2-n}^{n-2} \bigwedge_{\substack{0 \le i < i' < n \\ 0 \le j, j' < n \\ i - j = i' - j' = q}} \left(\neg B_{i,j} \vee \neg B_{i',j'} \right). \tag{25}$$

The condition $i < i'$ enumerates each unordered pair exactly once. Singleton diagonals contribute no clauses.

For $n = 4$, the central main diagonal $q = 0$ contains $B_{0,0}$, $B_{1,1}$, $B_{2,2}$, and $B_{3,3}$. Its AMO constraint is

$$\begin{aligned} F_{\text{main},0}^{\text{AMO}} = {} & (\neg B_{0,0} \vee \neg B_{1,1}) \\ & \wedge (\neg B_{0,0} \vee \neg B_{2,2}) \\ & \wedge (\neg B_{0,0} \vee \neg B_{3,3}) \\ & \wedge (\neg B_{1,1} \vee \neg B_{2,2}) \\ & \wedge (\neg B_{1,1} \vee \neg B_{3,3}) \\ & \wedge (\neg B_{2,2} \vee \neg B_{3,3}). \end{aligned} \tag{26}$$

Anti-diagonals and their parallels. Cells on the same anti-diagonal satisfy $i + j = k$. The non-singleton anti-diagonals have $k \in \{1, \dots, 2n - 3\}$:

$$F_{\text{anti}}^{\text{AMO}} = \bigwedge_{k=1}^{2n-3} \bigwedge_{\substack{0 \le i < i' < n \\ 0 \le j, j' < n \\ i + j = i' + j' = k}} \left(\neg B_{i,j} \vee \neg B_{i',j'} \right). \tag{27}$$

For $n = 4$, anti-diagonal $k = 2$ contains $B_{0,2}$, $B_{1,1}$, and $B_{2,0}$. Its three binary clauses are

$$\begin{aligned} F_{\text{anti},2}^{\text{AMO}} = {} & (\neg B_{0,2} \vee \neg B_{1,1}) \\ & \wedge (\neg B_{0,2} \vee \neg B_{2,0}) \\ & \wedge (\neg B_{1,1} \vee \neg B_{2,0}). \end{aligned} \tag{28}$$

The complete CNF is the conjunction of the row AMO and ALO, column AMO and ALO, and both diagonal AMO families. Every satisfying assignment therefore selects exactly n mutually non-attacking queens. Conversely, every valid placement satisfies all of these clauses.

3.2.3. Complexity Analysis

For a group of m variables, Pairwise encoding introduces zero auxiliary variables and generates one binary clause for each unordered pair:

$$A_{\text{binom}}(m) = 0, \qquad C_{\text{AMO}}(m) = \binom{m}{2} = \frac{m(m-1)}{2}. \tag{29}$$

Thus, the clause count of a single AMO constraint is $\Theta(m^2)$ as m grows. For example, a group of four variables requires $\binom{4}{2} = 6$ clauses, while a group of three requires $\binom{3}{2} = 3$ clauses, as demonstrated by the diagonal examples above. Groups containing zero or one variable generate no AMO clauses.

For the complete board, the total number of CNF variables is

$$V_{\text{binom}}(n) = n^2. \tag{30}$$

The n rows and n columns together contribute $2n\binom{n}{2}$ binary clauses. Within each diagonal family, one diagonal has length n, and two diagonals have each length m for $1 \le m < n$. Therefore, the two diagonal families contribute

$$\begin{aligned} C_{\text{diag}}(n) &= 2\binom{n}{2} + 4\sum_{m=1}^{n-1}\binom{m}{2} \\ &= 2\binom{n}{2} + 4\binom{n}{3} \\ &= \frac{n(n-1)(2n-1)}{3}. \end{aligned} \tag{31}$$

Including the $2n$ ALO clauses, the exact total is

$$\begin{aligned} C_{\text{binom}}(n) &= 2n\binom{n}{2} + 2\binom{n}{2} + 4\binom{n}{3} + 2n \\ &= \frac{5n^3 - 6n^2 + 7n}{3} \\ &= \Theta(n^3). \end{aligned} \tag{32}$$

This count refers to emitted clauses before preprocessing, with all row and column ALO clauses retained.

For $n = 4$, the clause counts are explicitly

$$\begin{aligned} V_{\text{binom}}(4) &= 16, \\ C^{\text{AMO}}_{\text{row}}(4) &= 4 \cdot 6 = 24, \\ C^{\text{AMO}}_{\text{col}}(4) &= 4 \cdot 6 = 24, \\ C_{\text{diag}}(4) &= 2 \cdot 6 + 4(0 + 1 + 3) = 28, \\ C^{\text{ALO}}(4) &= 4 + 4 = 8, \\ C_{\text{binom}}(4) &= 24 + 24 + 28 + 8 = 84. \end{aligned} \tag{33}$$

Hence, the 4×4 formulation contains 16 decision variables, no auxiliary variables, 76 binary AMO clauses, and eight ALO clauses of length four.

3.3. Binary Encoding

Binary encoding enforces an at-most-one constraint by assigning a distinct binary index to each input variable. A true input fixes the auxiliary bits to its index. Since distinct indices differ in at least one bit, two distinct inputs cannot both be true.

3.3.1. Encoding Variables

- **Decision variables.** The board is represented by

  $$B_{i,j} \in \{0, 1\}, \qquad 0 \le i, j < n, \tag{34}$$

  where $B_{i,j}$ is true exactly when a queen occupies cell (i, j).

- **Auxiliary variables.** Consider an ordered group $G = (X_0, \dots, X_{m-1})$ with $m \ge 2$. Introduce

  $$k_m = \lceil \log_2 m \rceil \tag{35}$$

  fresh Boolean variables $Y^G_0, \dots, Y^G_{k_m - 1}$. These variables encode the index of a selected input. Each row, column, and diagonal receives its own auxiliary variables; bits are never shared between different groups.

For an input index $t \in \{0, \dots, m-1\}$, define its p-th bit by

$$b_p(t) = \left\lfloor \frac{t}{2^p} \right\rfloor \bmod 2, \qquad 0 \le p < k_m. \tag{36}$$

Bit $p = 0$ is the least significant bit. The corresponding index literal is

$$L_{t,p}^{G} = \begin{cases} Y_p^G, & b_p(t) = 1, \\ \neg Y_p^G, & b_p(t) = 0. \end{cases} \tag{37}$$

For a group of four inputs, two bits suffice. The indices 0, 1, 2, 3 correspond respectively to bit pairs (Y_0^G, Y_1^G) equal to $(0,0)$, $(1,0)$, $(0,1)$, and $(1,1)$. The generic AMO construction is

$$E_{\text{bin}}(G) = \bigwedge_{t=0}^{m-1} \bigwedge_{p=0}^{k_m-1} \left(\neg X_t \vee L_{t,p}^{G} \right). \tag{38}$$

For $m \le 1$, the encoder generates neither auxiliary variables nor clauses.

3.3.2. Encoding Constraints

(a) Exactly One Queen Per Row. Let $R_i = (B_{i,0}, \dots, B_{i,n-1})$ and $k_n = \lceil \log_2 n \rceil$ for $n \ge 2$. Each row has an independent auxiliary vector Y^{R_i}.

At most one queen per row. The row AMO formula is

$$F_{\text{row,bin}}^{\text{AMO}} = \bigwedge_{i=0}^{n-1} \bigwedge_{j=0}^{n-1} \bigwedge_{p=0}^{k_n-1} \left(\neg B_{i,j} \vee L_{j,p}^{R_i} \right). \tag{39}$$

For $n = 4$, consider row 0 and abbreviate its two auxiliary variables as Y_0 and Y_1. The input-to-index implications are

$$\begin{aligned} &(B_{0,0} \to (\neg Y_0 \wedge \neg Y_1)) \\ \wedge\, &(B_{0,1} \to (Y_0 \wedge \neg Y_1)) \\ \wedge\, &(B_{0,2} \to (\neg Y_0 \wedge Y_1)) \\ \wedge\, &(B_{0,3} \to (Y_0 \wedge Y_1)). \end{aligned} \tag{40}$$

Their complete CNF expansion contains eight binary clauses:

$$\begin{aligned} F_{\text{row,0,bin}}^{\text{AMO}} = &(\neg B_{0,0} \vee \neg Y_0) \\ &\wedge (\neg B_{0,0} \vee \neg Y_1) \\ &\wedge (\neg B_{0,1} \vee Y_0) \\ &\wedge (\neg B_{0,1} \vee \neg Y_1) \\ &\wedge (\neg B_{0,2} \vee \neg Y_0) \\ &\wedge (\neg B_{0,2} \vee Y_1) \\ &\wedge (\neg B_{0,3} \vee Y_0) \\ &\wedge (\neg B_{0,3} \vee Y_1). \end{aligned} \tag{41}$$

At least one queen per row. The ALO construction is unchanged:

$$F_{\text{row}}^{\text{ALO}} = \bigwedge_{i=0}^{n-1} \left(\bigvee_{j=0}^{n-1} B_{i,j} \right). \tag{42}$$

For $n = 4$, all four row clauses are

$$\begin{aligned} F_{\text{row}}^{\text{ALO}} = &(B_{0,0} \vee B_{0,1} \vee B_{0,2} \vee B_{0,3}) \\ &\wedge (B_{1,0} \vee B_{1,1} \vee B_{1,2} \vee B_{1,3}) \\ &\wedge (B_{2,0} \vee B_{2,1} \vee B_{2,2} \vee B_{2,3}) \\ &\wedge (B_{3,0} \vee B_{3,1} \vee B_{3,2} \vee B_{3,3}). \end{aligned} \tag{43}$$

(b) Exactly One Queen Per Column. Let $C_j = (B_{0,j}, \ldots, B_{n-1,j})$. Each column receives its own auxiliary vector Y^{C_j}.

At most one queen per column. The column AMO formula is

$$F_{\text{col,bin}}^{\text{AMO}} = \bigwedge_{j=0}^{n-1} \bigwedge_{i=0}^{n-1} \bigwedge_{p=0}^{k_n-1} \left(\neg B_{i,j} \vee L_{i,p}^{C_j} \right). \tag{44}$$

For $n = 4$, column 0 uses two fresh variables Y'_0 and Y'_1, distinct from the row auxiliaries. Its eight binary clauses are

$$\begin{aligned} F_{\text{col,0,bin}}^{\text{AMO}} = {} & (\neg B_{0,0} \vee \neg Y'_0) \\ & \wedge (\neg B_{0,0} \vee \neg Y'_1) \\ & \wedge (\neg B_{1,0} \vee Y'_0) \\ & \wedge (\neg B_{1,0} \vee \neg Y'_1) \\ & \wedge (\neg B_{2,0} \vee \neg Y'_0) \\ & \wedge (\neg B_{2,0} \vee Y'_1) \\ & \wedge (\neg B_{3,0} \vee Y'_0) \\ & \wedge (\neg B_{3,0} \vee Y'_1). \end{aligned} \tag{45}$$

At least one queen per column. The column ALO formula is

$$F_{\text{col}}^{\text{ALO}} = \bigwedge_{j=0}^{n-1} \left(\bigvee_{i=0}^{n-1} B_{i,j} \right). \tag{46}$$

For $n = 4$, this expands into

$$\begin{aligned} F_{\text{col}}^{\text{ALO}} = {} & (B_{0,0} \vee B_{1,0} \vee B_{2,0} \vee B_{3,0}) \\ & \wedge (B_{0,1} \vee B_{1,1} \vee B_{2,1} \vee B_{3,1}) \\ & \wedge (B_{0,2} \vee B_{1,2} \vee B_{2,2} \vee B_{3,2}) \\ & \wedge (B_{0,3} \vee B_{1,3} \vee B_{2,3} \vee B_{3,3}). \end{aligned} \tag{47}$$

(c) At Most One Queen Per Diagonal. Diagonal inputs are ordered by increasing row index. Binary codes identify positions within each diagonal, rather than absolute board coordinates. Every diagonal has a separate auxiliary vector, and no diagonal ALO constraint is introduced.

Main diagonals and their parallels. For a diagonal indexed by $q = i - j$, define

$$\begin{aligned} a_q &= \max(0, q), \\ m_q &= n - |q|, \\ D_q &= \left(B_{a_q+t,\, a_q+t-q} \right)_{t=0}^{m_q-1}. \end{aligned} \tag{48}$$

The non-singleton main diagonals have $2 - n \le q \le n - 2$. Their AMO formula is

$$\begin{aligned} F_{\text{main,bin}}^{\text{AMO}} = {} & \bigwedge_{q=2-n}^{n-2} \bigwedge_{t=0}^{m_q-1} \bigwedge_{p=0}^{k_{m_q}-1} \\ & \left(\neg B_{a_q+t,\, a_q+t-q} \vee L_{t,p}^{D_q} \right). \end{aligned} \tag{49}$$

For $n = 4$ and $q = 0$, the ordered group is $(B_{0,0}, B_{1,1}, B_{2,2}, B_{3,3})$. Using fresh bits Z_0, Z_1, the complete expansion is

$$\begin{aligned} F^{\mathrm{AMO}}_{\mathrm{main},0,\mathrm{bin}} = {} & (\neg B_{0,0} \vee \neg Z_0) \\ & \wedge (\neg B_{0,0} \vee \neg Z_1) \\ & \wedge (\neg B_{1,1} \vee Z_0) \\ & \wedge (\neg B_{1,1} \vee \neg Z_1) \\ & \wedge (\neg B_{2,2} \vee \neg Z_0) \\ & \wedge (\neg B_{2,2} \vee Z_1) \\ & \wedge (\neg B_{3,3} \vee Z_0) \\ & \wedge (\neg B_{3,3} \vee Z_1). \end{aligned} \tag{50}$$

Anti-diagonals and their parallels. For an anti-diagonal indexed by $s = i + j$, define

$$\begin{aligned} a_s &= \max(0, s - n + 1), \\ m_s &= \min(n - 1, s) - a_s + 1, \\ A_s &= \left(B_{a_s + t,\, s - a_s - t} \right)_{t=0}^{m_s - 1}. \end{aligned} \tag{51}$$

The non-singleton anti-diagonals have $1 \le s \le 2n - 3$, giving

$$\begin{aligned} F^{\mathrm{AMO}}_{\mathrm{anti},\mathrm{bin}} = {} & \bigwedge_{s=1}^{2n-3} \bigwedge_{t=0}^{m_s - 1} \bigwedge_{p=0}^{k_{m_s} - 1} \\ & \left(\neg B_{a_s + t,\, s - a_s - t} \vee L^{A_s}_{t,p} \right). \end{aligned} \tag{52}$$

For $n = 4$ and $s = 2$, the ordered group is $(B_{0,2}, B_{1,1}, B_{2,0})$. Its length is three, so $\lceil \log_2 3 \rceil = 2$ fresh bits W_0, W_1 are required. The six binary clauses are

$$\begin{aligned} F^{\mathrm{AMO}}_{\mathrm{anti},2,\mathrm{bin}} = {} & (\neg B_{0,2} \vee \neg W_0) \\ & \wedge (\neg B_{0,2} \vee \neg W_1) \\ & \wedge (\neg B_{1,1} \vee W_0) \\ & \wedge (\neg B_{1,1} \vee \neg W_1) \\ & \wedge (\neg B_{2,0} \vee \neg W_0) \\ & \wedge (\neg B_{2,0} \vee W_1). \end{aligned} \tag{53}$$

The unused index 3, represented by $(W_0, W_1) = (1, 1)$, requires no additional exclusion clause. Under that bit assignment, all three diagonal inputs are forced false, which is valid for AMO. More generally, when all inputs are false, every assignment to the auxiliary bits satisfies the linking clauses. For rows and columns, the separate ALO clause guarantees that one input is true and therefore selects a valid index.

Correctness follows from the distinctness of the binary indices. If two inputs are true, a bit on which their indices differ is forced both true and false. Conversely, a single true input is accommodated by assigning its index to the auxiliary vector, and the all-false input assignment always has a satisfying extension. Thus, projecting the CNF solutions onto the input variables yields exactly AMO.

3.3.3. Complexity Analysis

For a group of $m \ge 2$ inputs, Binary encoding uses

$$\begin{aligned} A_{\mathrm{bin}}(m) &= \lceil \log_2 m \rceil, \\ C_{\mathrm{AMO},\mathrm{bin}}(m) &= m \lceil \log_2 m \rceil. \end{aligned} \tag{54}$$

All these clauses are binary. A group of four inputs therefore requires two auxiliary variables and eight clauses, while a group of three requires two auxiliary variables and six clauses.

For the complete board, there are $2n$ row and column groups of length n, together with two central diagonals of length n. Across the two diagonal families, each length m with $2 \le m < n$ occurs four times. Consequently, for $n \ge 2$, the exact auxiliary-variable count is

$$A_{\text{bin}}^{\text{board}}(n) = (2n+2)\lceil \log_2 n \rceil + 4 \sum_{m=2}^{n-1} \lceil \log_2 m \rceil . \tag{55}$$

It follows that

$$\begin{aligned} A_{\text{bin}}^{\text{board}}(n) &= \Theta(n \log_2 n), \\ V_{\text{bin}}(n) &= n^2 + A_{\text{bin}}^{\text{board}}(n) \\ &= \Theta(n^2). \end{aligned} \tag{56}$$

Including all $2n$ row and column ALO clauses, the exact clause count is

$$\begin{aligned} C_{\text{bin}}(n) &= 2n + (2n+2)n \lceil \log_2 n \rceil \\ &\quad + 4 \sum_{m=2}^{n-1} m \lceil \log_2 m \rceil \\ &= \Theta(n^2 \log_2 n). \end{aligned} \tag{57}$$

The lower bound already follows from the row and column AMO clauses; the diagonal contribution has the same asymptotic upper bound.

For $n = 4$, these expressions give

$$\begin{aligned} A_{\text{bin}}^{\text{board}}(4) &= 10 \cdot 2 + 4(1+2) = 32, \\ V_{\text{bin}}(4) &= 16 + 32 = 48, \\ C_{\text{bin}}(4) &= 8 + 10 \cdot 4 \cdot 2 + 4(2+6) \\ &= 120. \end{aligned} \tag{58}$$

Thus, the 4×4 instance contains 16 board variables, 32 auxiliary variables, 112 binary linking clauses, and eight ALO clauses of length four. At $n = 1$, the implementation instead generates one board variable, no auxiliary variables, and two identical positive unit clauses for the row and column ALO constraints.

3.4. Sequential Counter Encoding

Sequential Counter encoding enforces an at-most-one constraint through a chain of auxiliary variables that propagates prefix occupancy information. Once an input is true, the chain prevents every subsequent input from becoming true. This construction uses a linear number of auxiliary variables and binary clauses.

3.4.1. Encoding Variables

- **Decision variables.** For an $n \times n$ board, introduce

  $$B_{i,j} \in \{0,1\}, \qquad 0 \le i, j < n, \tag{59}$$

  where $B_{i,j}$ is true exactly when a queen occupies cell (i, j).

- **Auxiliary variables.** For an ordered group $G = (X_0, \dots, X_{m-1})$ with $m \ge 2$, introduce $m - 1$ fresh Boolean variables $S_0^G, \dots, S_{m-2}^G$. Each row, column, and diagonal has its own auxiliary chain; variables are never shared between groups.

The auxiliary variable S_t^G carries prefix occupancy information. The implemented CNF enforces

$$\left(\bigvee_{r=0}^{t} X_r \right) \to S_t^G, \qquad 0 \le t \le m-2. \tag{60}$$

Assigning $S_t^G = \bigvee_{r=0}^{t} X_r$ provides a canonical extension of every input assignment satisfying AMO. The CNF uses the forward implication rather than an equivalence: auxiliary assignments need not be unique. For example, with four false inputs, setting $S_0^G = S_1^G = S_2^G = \text{true}$ also satisfies the encoding.

3.4.2. Encoding Constraints

For a group $G = (X_0, \dots, X_{m-1})$ with $m \geq 2$, define the sequential AMO encoding as

$$\begin{aligned} E_{\text{seq}}(G) = {} & (\neg X_0 \vee S_0^G) \\ & \wedge (\neg S_{m-2}^G \vee \neg X_{m-1}) \\ & \wedge \bigwedge_{t=1}^{m-2} \Big[(\neg X_t \vee S_t^G) \\ & \quad \wedge (\neg S_{t-1}^G \vee S_t^G) \\ & \quad \wedge (\neg S_{t-1}^G \vee \neg X_t) \Big]. \end{aligned} \tag{61}$$

The first clause initializes the chain, the middle clauses propagate occupancy and exclude a second true input, and the final clause checks the last input. At $m = 2$, the middle conjunction is empty and only the two boundary clauses remain. Groups of length zero or one generate neither auxiliary variables nor clauses.

(a) Exactly One Queen Per Row. Let $R_i = (B_{i,0}, \dots, B_{i,n-1})$ and denote its auxiliary variables by $S_{i,t} = S_t^{R_i}$.

At most one queen per row. For $n \geq 2$, applying the sequential construction to every row gives

$$\begin{aligned} F_{\text{row,seq}}^{\text{AMO}} = \bigwedge_{i=0}^{n-1} \Big[& (\neg B_{i,0} \vee S_{i,0}) \\ & \wedge (\neg S_{i,n-2} \vee \neg B_{i,n-1}) \\ & \wedge \bigwedge_{t=1}^{n-2} \Big[(\neg B_{i,t} \vee S_{i,t}) \\ & \quad \wedge (\neg S_{i,t-1} \vee S_{i,t}) \\ & \quad \wedge (\neg S_{i,t-1} \vee \neg B_{i,t}) \Big] \Big]. \end{aligned} \tag{62}$$

For $n = 4$, row 0 uses three auxiliary variables $S_{0,0}, S_{0,1}, S_{0,2}$. Its logical implications are

$$\begin{aligned} & (B_{0,0} \to S_{0,0}) \\ \wedge\ & ((B_{0,1} \vee S_{0,0}) \to S_{0,1}) \\ \wedge\ & (S_{0,0} \to \neg B_{0,1}) \\ \wedge\ & ((B_{0,2} \vee S_{0,1}) \to S_{0,2}) \\ \wedge\ & (S_{0,1} \to \neg B_{0,2}) \\ \wedge\ & (S_{0,2} \to \neg B_{0,3}). \end{aligned} \tag{63}$$

Expanding these implications yields exactly eight binary clauses:

$$\begin{aligned} F_{\text{row,0,seq}}^{\text{AMO}} = {} & (\neg B_{0,0} \vee S_{0,0}) \\ & \wedge (\neg B_{0,1} \vee S_{0,1}) \\ & \wedge (\neg S_{0,0} \vee S_{0,1}) \\ & \wedge (\neg S_{0,0} \vee \neg B_{0,1}) \\ & \wedge (\neg B_{0,2} \vee S_{0,2}) \\ & \wedge (\neg S_{0,1} \vee S_{0,2}) \\ & \wedge (\neg S_{0,1} \vee \neg B_{0,2}) \\ & \wedge (\neg S_{0,2} \vee \neg B_{0,3}). \end{aligned} \tag{64}$$

At least one queen per row. Each row also receives one positive clause:

$$F_{\text{row}}^{\text{ALO}} = \bigwedge_{i=0}^{n-1} \left(\bigvee_{j=0}^{n-1} B_{i,j} \right). \tag{65}$$

For $n = 4$, the complete row ALO formula is

$$\begin{aligned} F_{\text{row}}^{\text{ALO}} = {} & (B_{0,0} \vee B_{0,1} \vee B_{0,2} \vee B_{0,3}) \\ & \wedge (B_{1,0} \vee B_{1,1} \vee B_{1,2} \vee B_{1,3}) \\ & \wedge (B_{2,0} \vee B_{2,1} \vee B_{2,2} \vee B_{2,3}) \\ & \wedge (B_{3,0} \vee B_{3,1} \vee B_{3,2} \vee B_{3,3}). \end{aligned} \tag{66}$$

(b) Exactly One Queen Per Column. Let $C_j = (B_{0,j}, \dots, B_{n-1,j})$ and denote its fresh auxiliary variables by $T_{j,t} = S_t^{C_j}$.

At most one queen per column. For $n \geq 2$, the column AMO formula is

$$\begin{aligned} F_{\text{col,seq}}^{\text{AMO}} = \bigwedge_{j=0}^{n-1} \Big[& (\neg B_{0,j} \vee T_{j,0}) \\ & \wedge (\neg T_{j,n-2} \vee \neg B_{n-1,j}) \\ & \wedge \bigwedge_{t=1}^{n-2} \Big[(\neg B_{t,j} \vee T_{j,t}) \\ & \qquad \wedge (\neg T_{j,t-1} \vee T_{j,t}) \\ & \qquad \wedge (\neg T_{j,t-1} \vee \neg B_{t,j}) \Big] \Big]. \end{aligned} \tag{67}$$

For $n = 4$, column 0 uses $T_{0,0}, T_{0,1}, T_{0,2}$ and produces

$$\begin{aligned} F_{\text{col,0,seq}}^{\text{AMO}} = {} & (\neg B_{0,0} \vee T_{0,0}) \\ & \wedge (\neg B_{1,0} \vee T_{0,1}) \\ & \wedge (\neg T_{0,0} \vee T_{0,1}) \\ & \wedge (\neg T_{0,0} \vee \neg B_{1,0}) \\ & \wedge (\neg B_{2,0} \vee T_{0,2}) \\ & \wedge (\neg T_{0,1} \vee T_{0,2}) \\ & \wedge (\neg T_{0,1} \vee \neg B_{2,0}) \\ & \wedge (\neg T_{0,2} \vee \neg B_{3,0}). \end{aligned} \tag{68}$$

At least one queen per column. The column ALO formula is

$$F_{\text{col}}^{\text{ALO}} = \bigwedge_{j=0}^{n-1} \left(\bigvee_{i=0}^{n-1} B_{i,j} \right). \tag{69}$$

For $n = 4$, its four clauses are

$$\begin{aligned} F_{\text{col}}^{\text{ALO}} = {} & (B_{0,0} \vee B_{1,0} \vee B_{2,0} \vee B_{3,0}) \\ & \wedge (B_{0,1} \vee B_{1,1} \vee B_{2,1} \vee B_{3,1}) \\ & \wedge (B_{0,2} \vee B_{1,2} \vee B_{2,2} \vee B_{3,2}) \\ & \wedge (B_{0,3} \vee B_{1,3} \vee B_{2,3} \vee B_{3,3}). \end{aligned} \tag{70}$$

(c) At Most One Queen Per Diagonal. Diagonal cells are ordered by increasing row index. Each non-singleton diagonal receives an independent sequential chain. No diagonal ALO clause is added.

Main diagonals and their parallels. For a main diagonal indexed by $q = i - j$, define

$$\begin{aligned} a_q &= \max(0, q), \\ m_q &= n - |q|, \\ D_q &= \left(B_{a_q+t,\, a_q+t-q}\right)_{t=0}^{m_q-1}. \end{aligned} \tag{71}$$

Using the explicit CNF construction in Equation (61), the main-diagonal formula is

$$F_{\text{main,seq}}^{\text{AMO}} = \bigwedge_{q=2-n}^{n-2} E_{\text{seq}}(D_q). \tag{72}$$

For $n = 4$ and $q = 0$, the group is $(B_{0,0}, B_{1,1}, B_{2,2}, B_{3,3})$. With three fresh auxiliary variables U_0, U_1, U_2, its eight clauses are

$$\begin{aligned} F_{\text{main,0,seq}}^{\text{AMO}} = {} & (\neg B_{0,0} \vee U_0) \\ & \wedge (\neg B_{1,1} \vee U_1) \\ & \wedge (\neg U_0 \vee U_1) \\ & \wedge (\neg U_0 \vee \neg B_{1,1}) \\ & \wedge (\neg B_{2,2} \vee U_2) \\ & \wedge (\neg U_1 \vee U_2) \\ & \wedge (\neg U_1 \vee \neg B_{2,2}) \\ & \wedge (\neg U_2 \vee \neg B_{3,3}). \end{aligned} \tag{73}$$

Anti-diagonals and their parallels. For an anti-diagonal indexed by $s = i + j$, define

$$\begin{aligned} a_s &= \max(0, s - n + 1), \\ m_s &= \min(n - 1, s) - a_s + 1, \\ A_s &= \left(B_{a_s+t,\, s-a_s-t}\right)_{t=0}^{m_s-1}. \end{aligned} \tag{74}$$

The anti-diagonal formula is

$$F_{\text{anti,seq}}^{\text{AMO}} = \bigwedge_{s=1}^{2n-3} E_{\text{seq}}(A_s). \tag{75}$$

For $n = 4$ and $s = 2$, the group is $(B_{0,2}, B_{1,1}, B_{2,0})$. Since $m = 3$, it requires two fresh auxiliary variables W_0, W_1 and five binary clauses:

$$\begin{aligned} F_{\text{anti,2,seq}}^{\text{AMO}} = {} & (\neg B_{0,2} \vee W_0) \\ & \wedge (\neg B_{1,1} \vee W_1) \\ & \wedge (\neg W_0 \vee W_1) \\ & \wedge (\neg W_0 \vee \neg B_{1,1}) \\ & \wedge (\neg W_1 \vee \neg B_{2,0}). \end{aligned} \tag{76}$$

To establish correctness, suppose two inputs X_a and X_b are true with $a < b$. The linking clauses force S_a^G true and propagate this value through the chain to S_{b-1}^G. The exclusion clause $(\neg S_{b-1}^G \vee \neg X_b)$ then yields a contradiction. Conversely, an input assignment with at most one true variable satisfies the encoding when each auxiliary variable is assigned the corresponding prefix disjunction. Therefore, the encoding represents AMO exactly under projection onto the input variables.

3.4.3. Complexity Analysis

For a group of $m \geq 2$ inputs, the encoding introduces $m - 1$ auxiliary variables. It contains two boundary clauses and three clauses for each of the $m - 2$ intermediate positions:

$$\begin{aligned} A_{\text{seq}}(m) &= m - 1, \\ C_{\text{AMO,seq}}(m) &= 2 + 3(m - 2) = 3m - 4. \end{aligned} \tag{77}$$

Both quantities are $\Theta(m)$, and every AMO clause is binary. For $m = 4$, the encoding uses three auxiliary variables and eight clauses; for $m = 3$, it uses two auxiliary variables and five clauses.

For an $n \times n$ board with $n \geq 2$, the rows and columns contribute $2n$ groups of length n. The two diagonal families contribute two additional groups of length n and four groups of each length m with $2 \leq m < n$. Thus,

$$\begin{aligned} A_{\text{seq}}^{\text{board}}(n) &= (2n + 2)(n - 1) + 4 \sum_{m=2}^{n-1} (m - 1) \\ &= (2n + 2)(n - 1) + 2(n - 2)(n - 1) \\ &= 4n^2 - 6n + 2. \end{aligned} \tag{78}$$

Adding the n^2 board variables gives

$$\begin{aligned} V_{\text{seq}}(n) &= n^2 + A_{\text{seq}}^{\text{board}}(n) \\ &= 5n^2 - 6n + 2 \\ &= \Theta(n^2). \end{aligned} \tag{79}$$

Including the $2n$ ALO clauses for rows and columns, the complete clause count is

$$\begin{aligned} C_{\text{seq}}(n) &= 2n + (2n + 2)(3n - 4) + 4 \sum_{m=2}^{n-1} (3m - 4) \\ &= 2n + (2n + 2)(3n - 4) + 2(n - 2)(3n - 5) \\ &= 12n^2 - 22n + 12 \\ &= \Theta(n^2). \end{aligned} \tag{80}$$

These counts describe the emitted CNF before preprocessing and retain all row and column ALO clauses.

For $n = 4$, the exact counts are

$$\begin{aligned} A_{\text{seq}}^{\text{board}}(4) &= 10 \cdot 3 + 4(1 + 2) = 42, \\ V_{\text{seq}}(4) &= 16 + 42 = 58, \\ C_{\text{seq}}(4) &= 8 + 10 \cdot 8 + 4(2 + 5) \\ &= 116. \end{aligned} \tag{81}$$

The resulting CNF contains 16

3.5. Commander Encoding

Commander encoding enforces an at-most-one constraint hierarchically. Inputs are partitioned into small groups, each represented by a commander variable, and the same encoding is applied recursively to the commanders. The implementation uses consecutive groups of three, with a possibly shorter final group, and switches directly to Pairwise encoding when at most three inputs remain. Accordingly, four inputs are partitioned as $3 + 1$; the examples below follow this implemented construction.

3.5.1. Encoding Variables

- **Decision variables.** The board variables are

$$B_{i,j} \in \{0,1\}, \qquad 0 \le i,j < n, \tag{82}$$

where $B_{i,j}$ is true exactly when a queen occupies cell (i,j).

- **Auxiliary variables.** For an ordered input sequence $X = (X_0, \dots, X_{m-1})$ with $m > 3$, define

$$\begin{aligned} g &= \left\lceil \frac{m}{3} \right\rceil, \\ G_t &= \{X_r : 3(t-1) \le r < \min(3t, m)\}, \\ & \qquad\qquad 1 \le t \le g. \end{aligned} \tag{83}$$

Introduce a fresh commander c_t for each group, with the exact semantics

$$c_t \leftrightarrow \bigvee_{X_r \in G_t} X_r. \tag{84}$$

Recursive calls allocate additional fresh commanders. Different rows, columns, diagonals, and recursion levels never share auxiliary variables.

For $m \le 3$, no commander variables are introduced. The input group is encoded directly by its pairwise exclusion clauses.

3.5.2. Encoding Constraints

Let $E_{\text{pw}}(G)$ denote the conjunction of all pairwise exclusion clauses within a group G. For $m > 3$, Commander encoding combines three components:

$$\begin{aligned} E_{\text{com}}(X) = \bigwedge_{t=1}^{g} \Bigg[& E_{\text{pw}}(G_t) \\ & \wedge \bigwedge_{X_r \in G_t} (\neg X_r \vee c_t) \\ & \wedge \left(\neg c_t \vee \bigvee_{X_r \in G_t} X_r \right) \Bigg] \\ & \wedge E_{\text{com}}(c_1, \dots, c_g). \end{aligned} \tag{85}$$

- **Local AMO.** $E_{\text{pw}}(G_t)$ prevents two inputs in the same group from being true.
- **Bidirectional linking.** Each clause $(\neg X_r \vee c_t)$ activates the commander when an input is true. The reverse clause $(\neg c_t \vee \bigvee_{X_r \in G_t} X_r)$ requires an active commander to have a true input in its group.
- **Recursive AMO.** $E_{\text{com}}(c_1, \dots, c_g)$ prevents two different groups from being simultaneously active.

The recursion terminates with

$$E_{\text{com}}(X) = E_{\text{pw}}(X), \qquad m \le 3. \tag{86}$$

For zero or one input, the pairwise conjunction is empty and no clauses are generated.

(a) Exactly One Queen Per Row. Let $R_i = (B_{i,0}, \ldots, B_{i,n-1})$. The row AMO constraints are

$$F_{\text{row,com}}^{\text{AMO}} = \bigwedge_{i=0}^{n-1} E_{\text{com}}(R_i). \tag{87}$$

For $n = 4$, row 0 is partitioned into

$$\begin{aligned} G_1 &= \{B_{0,0}, B_{0,1}, B_{0,2}\}, \\ G_2 &= \{B_{0,3}\}. \end{aligned} \tag{88}$$

Its commanders satisfy $c_1 \leftrightarrow (B_{0,0} \vee B_{0,1} \vee B_{0,2})$ and $c_2 \leftrightarrow B_{0,3}$. The logical implications can be written as

$$\begin{aligned} &(B_{0,0} \rightarrow \neg B_{0,1}) \\ \wedge\, &(B_{0,0} \rightarrow \neg B_{0,2}) \\ \wedge\, &(B_{0,1} \rightarrow \neg B_{0,2}) \\ \wedge\, &(B_{0,0} \rightarrow c_1) \\ \wedge\, &(B_{0,1} \rightarrow c_1) \\ \wedge\, &(B_{0,2} \rightarrow c_1) \\ \wedge\, &(c_1 \rightarrow (B_{0,0} \vee B_{0,1} \vee B_{0,2})) \\ \wedge\, &(B_{0,3} \rightarrow c_2) \\ \wedge\, &(c_2 \rightarrow B_{0,3}) \\ \wedge\, &(c_1 \rightarrow \neg c_2). \end{aligned} \tag{89}$$

The complete CNF contains ten clauses:

$$\begin{aligned} F_{\text{row,0,com}}^{\text{AMO}} = &(\neg B_{0,0} \vee \neg B_{0,1}) \\ &\wedge (\neg B_{0,0} \vee \neg B_{0,2}) \\ &\wedge (\neg B_{0,1} \vee \neg B_{0,2}) \\ &\wedge (\neg B_{0,0} \vee c_1) \\ &\wedge (\neg B_{0,1} \vee c_1) \\ &\wedge (\neg B_{0,2} \vee c_1) \\ &\wedge (\neg c_1 \vee B_{0,0} \vee B_{0,1} \vee B_{0,2}) \\ &\wedge (\neg B_{0,3} \vee c_2) \\ &\wedge (\neg c_2 \vee B_{0,3}) \\ &\wedge (\neg c_1 \vee \neg c_2). \end{aligned} \tag{90}$$

The final clause is the Pairwise base case on the two commanders. A balanced $2 + 2$ partition would produce nine clauses, but constitutes a different grouping choice from the implemented consecutive $3 + 1$ partition.

At least one queen per row. The row ALO formula is

$$F_{\text{row}}^{\text{ALO}} = \bigwedge_{i=0}^{n-1} \left(\bigvee_{j=0}^{n-1} B_{i,j} \right). \tag{91}$$

For $n = 4$, it expands into

$$\begin{aligned} F_{\text{row}}^{\text{ALO}} = &(B_{0,0} \vee B_{0,1} \vee B_{0,2} \vee B_{0,3}) \\ &\wedge (B_{1,0} \vee B_{1,1} \vee B_{1,2} \vee B_{1,3}) \\ &\wedge (B_{2,0} \vee B_{2,1} \vee B_{2,2} \vee B_{2,3}) \\ &\wedge (B_{3,0} \vee B_{3,1} \vee B_{3,2} \vee B_{3,3}). \end{aligned} \tag{92}$$

(b) Exactly One Queen Per Column. Let $C_j = (B_{0,j}, \dots, B_{n-1,j})$. The column AMO formula is

$$F_{\text{col,com}}^{\text{AMO}} = \bigwedge_{j=0}^{n-1} E_{\text{com}}(C_j). \tag{93}$$

For $n = 4$, column 0 is partitioned into

$$\begin{aligned} G'_1 &= \{B_{0,0}, B_{1,0}, B_{2,0}\}, \\ G'_2 &= \{B_{3,0}\}, \end{aligned} \tag{94}$$

with fresh commanders c'_1, c'_2. The ten clauses are

$$\begin{aligned} F_{\text{col,0,com}}^{\text{AMO}} = {} & (\neg B_{0,0} \vee \neg B_{1,0}) \\ & \wedge (\neg B_{0,0} \vee \neg B_{2,0}) \\ & \wedge (\neg B_{1,0} \vee \neg B_{2,0}) \\ & \wedge (\neg B_{0,0} \vee c'_1) \\ & \wedge (\neg B_{1,0} \vee c'_1) \\ & \wedge (\neg B_{2,0} \vee c'_1) \\ & \wedge (\neg c'_1 \vee B_{0,0} \vee B_{1,0} \vee B_{2,0}) \\ & \wedge (\neg B_{3,0} \vee c'_2) \\ & \wedge (\neg c'_2 \vee B_{3,0}) \\ & \wedge (\neg c'_1 \vee \neg c'_2). \end{aligned} \tag{95}$$

At least one queen per column. The column ALO formula is

$$F_{\text{col}}^{\text{ALO}} = \bigwedge_{j=0}^{n-1} \left(\bigvee_{i=0}^{n-1} B_{i,j} \right). \tag{96}$$

For $n = 4$, the four clauses are

$$\begin{aligned} F_{\text{col}}^{\text{ALO}} = {} & (B_{0,0} \vee B_{1,0} \vee B_{2,0} \vee B_{3,0}) \\ & \wedge (B_{0,1} \vee B_{1,1} \vee B_{2,1} \vee B_{3,1}) \\ & \wedge (B_{0,2} \vee B_{1,2} \vee B_{2,2} \vee B_{3,2}) \\ & \wedge (B_{0,3} \vee B_{1,3} \vee B_{2,3} \vee B_{3,3}). \end{aligned} \tag{97}$$

(c) At Most One Queen Per Diagonal. Each diagonal is ordered by increasing row index and encoded independently. No diagonal ALO clause is added.

Main diagonals and their parallels. For $q = i - j$, define

$$\begin{aligned} a_q &= \max(0, q), \\ m_q &= n - |q|, \\ D_q &= \left(B_{a_q+t,\, a_q+t-q} \right)_{t=0}^{m_q-1}. \end{aligned} \tag{98}$$

The main-diagonal formula is

$$F_{\text{main,com}}^{\text{AMO}} = \bigwedge_{q=2-n}^{n-2} E_{\text{com}}(D_q). \tag{99}$$

For $n = 4$ and $q = 0$, partition $(B_{0,0}, B_{1,1}, B_{2,2}, B_{3,3})$ into a group of three and a singleton, represented by fresh commanders d_1, d_2. The clauses are

$$\begin{aligned} F^{\mathrm{AMO}}_{\mathrm{main},0,\mathrm{com}} = {} & (\neg B_{0,0} \vee \neg B_{1,1}) \\ & \wedge (\neg B_{0,0} \vee \neg B_{2,2}) \\ & \wedge (\neg B_{1,1} \vee \neg B_{2,2}) \\ & \wedge (\neg B_{0,0} \vee d_1) \\ & \wedge (\neg B_{1,1} \vee d_1) \\ & \wedge (\neg B_{2,2} \vee d_1) \\ & \wedge (\neg d_1 \vee B_{0,0} \vee B_{1,1} \vee B_{2,2}) \\ & \wedge (\neg B_{3,3} \vee d_2) \\ & \wedge (\neg d_2 \vee B_{3,3}) \\ & \wedge (\neg d_1 \vee \neg d_2). \end{aligned} \tag{100}$$

Anti-diagonals and their parallels. For $s = i + j$, define

$$\begin{aligned} a_s &= \max(0, s - n + 1), \\ m_s &= \min(n - 1, s) - a_s + 1, \\ A_s &= \left(B_{a_s+t,\, s-a_s-t}\right)_{t=0}^{m_s-1}. \end{aligned} \tag{101}$$

The anti-diagonal formula is

$$F^{\mathrm{AMO}}_{\mathrm{anti,com}} = \bigwedge_{s=1}^{2n-3} E_{\mathrm{com}}(A_s). \tag{102}$$

For $n = 4$ and $s = 2$, the inputs are $(B_{0,2}, B_{1,1}, B_{2,0})$. Since the group contains only three inputs, the Pairwise base case applies directly:

$$\begin{aligned} F^{\mathrm{AMO}}_{\mathrm{anti},2,\mathrm{com}} = {} & (\neg B_{0,2} \vee \neg B_{1,1}) \\ & \wedge (\neg B_{0,2} \vee \neg B_{2,0}) \\ & \wedge (\neg B_{1,1} \vee \neg B_{2,0}). \end{aligned} \tag{103}$$

This diagonal therefore uses zero auxiliary variables and exactly three binary clauses.

Correctness follows recursively. Two true inputs in the same group violate its local Pairwise constraint. Two true inputs in different groups activate two commanders and violate the recursively encoded AMO. Conversely, any input assignment satisfying AMO extends to a satisfying assignment by setting every commander to the disjunction of its group.

3.5.3. Complexity Analysis

Let $a_{\mathrm{com}}(m)$ and $c_{\mathrm{com}}(m)$ denote the auxiliary-variable and clause counts for one AMO constraint on m inputs. The base cases are

$$a_{\mathrm{com}}(m) = 0, \qquad c_{\mathrm{com}}(m) = \binom{m}{2}, \qquad 0 \le m \le 3. \tag{104}$$

In particular, $a_{\mathrm{com}}(3) = 0$ and $c_{\mathrm{com}}(3) = 3$.

For $m > 3$, let $g = \lceil m/3 \rceil$ and $b_t = |G_t|$. Each group contributes $\binom{b_t}{2}$ local exclusion clauses, b_t forward linking clauses, and one reverse clause. The exact recurrences are

$$\begin{aligned} a_{\mathrm{com}}(m) &= g + a_{\mathrm{com}}(g), \\ c_{\mathrm{com}}(m) &= \sum_{t=1}^{g} \left[\binom{b_t}{2} + b_t + 1 \right] + c_{\mathrm{com}}(g). \end{aligned} \tag{105}$$

For $m = 4$, these give

$$\begin{aligned} a_{\mathrm{com}}(4) &= 2, \\ c_{\mathrm{com}}(4) &= (3+3+1) + (0+1+1) + 1 \\ &= 10. \end{aligned} \tag{106}$$

When $m = 3^h$ with $h \geq 1$, every group above the base case has exactly three inputs. Consequently,

$$\begin{aligned} a_{\mathrm{com}}(3^h) &= \frac{3^h - 3}{2}, \\ c_{\mathrm{com}}(3^h) &= \frac{7 \cdot 3^h - 15}{2}. \end{aligned} \tag{107}$$

For arbitrary m, rounding and the final partial groups give

$$\begin{aligned} a_{\mathrm{com}}(m) &= \frac{m}{2} + O(\log m), \\ c_{\mathrm{com}}(m) &= \frac{7m}{2} + O(\log m). \end{aligned} \tag{108}$$

Both counts are therefore $\Theta(m)$ as m grows.

For an $n \times n$ board with $n \geq 2$, the exact auxiliary-variable count is

$$\begin{aligned} A_{\mathrm{com}}^{\mathrm{board}}(n) &= (2n+2)a_{\mathrm{com}}(n) \\ &\quad + 4 \sum_{m=2}^{n-1} a_{\mathrm{com}}(m), \end{aligned} \tag{109}$$

and the total CNF counts are

$$\begin{aligned} V_{\mathrm{com}}(n) &= n^2 + A_{\mathrm{com}}^{\mathrm{board}}(n), \\ C_{\mathrm{com}}(n) &= 2n + (2n+2)c_{\mathrm{com}}(n) \\ &\quad + 4 \sum_{m=2}^{n-1} c_{\mathrm{com}}(m). \end{aligned} \tag{110}$$

The term $2n$ counts the row and column ALO clauses. Substituting the group asymptotics yields

$$\begin{aligned} A_{\mathrm{com}}^{\mathrm{board}}(n) &= 2n^2 + O(n \log n), \\ V_{\mathrm{com}}(n) &= 3n^2 + O(n \log n) = \Theta(n^2), \\ C_{\mathrm{com}}(n) &= 14n^2 + O(n \log n) = \Theta(n^2). \end{aligned} \tag{111}$$

Thus, the leading term $3n^2$ describes the *total* variable count, including board variables; the auxiliary-variable count has leading term $2n^2$.

For $n = 4$, the exact counts are

$$\begin{aligned} A_{\mathrm{com}}^{\mathrm{board}}(4) &= 10 \cdot 2 + 4(0+0) = 20, \\ V_{\mathrm{com}}(4) &= 16 + 20 = 36, \\ C_{\mathrm{com}}(4) &= 8 + 10 \cdot 10 + 4(1+3) \\ &= 124. \end{aligned} \tag{112}$$

The recursive hierarchy has depth $O(\log_3 m)$. A true input propagates upward to its commanders; exclusion at a common ancestor propagates downward to competing groups. The linking clauses also propagate a false commander to every input in its group, and the reverse clause forces a commander false when all its inputs are false. Together with the local Pairwise constraints, this structure ensures that setting one input true causes unit propagation to set every other input of the AMO constraint false.

3.6. Product Encoding

Product encoding represents an at-most-one constraint through two coordinate sets on an auxiliary grid. Each input activates one row coordinate and one column coordinate. At-most-one constraints on both coordinate sets prevent two distinct inputs from being simultaneously true.

The implemented variant is recursive: Product encoding is applied to the coordinate variables themselves. Groups containing at most four inputs are encoded directly using Pairwise clauses. This base case is included explicitly in the definitions, examples, and counting recurrences below.

3.6.1. Encoding Variables

- **Decision variables.** Introduce

$$B_{i,j} \in \{0,1\}, \qquad 0 \le i,j < n, \tag{113}$$

where $B_{i,j}$ is true exactly when a queen occupies cell (i,j).

- **Auxiliary variables.** For an ordered group $X = (X_0, \dots, X_{m-1})$ with $m > 4$, define

$$p = \left\lceil \sqrt{m} \right\rceil, \qquad q = \left\lceil \frac{m}{p} \right\rceil. \tag{114}$$

Introduce fresh row-coordinate variables $r_1, \dots, r_p$ and column-coordinate variables $c_1, \dots, c_q$. The input at index t is assigned to grid position

$$u(t) = \left\lfloor \frac{t}{q} \right\rfloor + 1, \qquad v(t) = (t \bmod q) + 1. \tag{115}$$

Every group and recursive call uses distinct auxiliary variables.

The grid contains at least m positions because $pq \ge m$. Only the first m positions are associated with inputs; unused positions introduce no additional input variables.

For example, a conceptual 2×2 grid maps four inputs as follows:

$$\begin{aligned} &X_0 \mapsto (1,1), \quad X_1 \mapsto (1,2), \\ &X_2 \mapsto (2,1), \quad X_3 \mapsto (2,2). \end{aligned} \tag{116}$$

This mapping illustrates the Product construction. Under the implemented cutoff, however, a group of four inputs is handled by Pairwise directly and allocates no coordinate variables.

3.6.2. Encoding Constraints

For $m > 4$, define

$$\begin{aligned} E_{\text{prod}}(X) = {} & E_{\text{prod}}(r_1, \dots, r_p) \\ & \wedge E_{\text{prod}}(c_1, \dots, c_q) \\ & \wedge \bigwedge_{t=0}^{m-1} \Big[(\neg X_t \vee r_{u(t)}) \\ & \qquad \wedge (\neg X_t \vee c_{v(t)}) \Big]. \end{aligned} \tag{117}$$

The base case is

$$E_{\text{prod}}(X) = \bigwedge_{0 \le a < b < m} (\neg X_a \vee \neg X_b), \qquad m \le 4. \tag{118}$$

For $m \le 1$, this conjunction is empty.

The linking clauses are exclusively forward implications:

$$X_t \to r_{u(t)}, \qquad X_t \to c_{v(t)}. \tag{119}$$

No reverse implication from a coordinate variable to a disjunction of inputs is required.

To establish correctness, consider two distinct true inputs. Their grid positions differ in a row coordinate, a column coordinate, or both. Their linking clauses therefore activate two distinct coordinates in at least one coordinate set, violating its recursively encoded AMO constraint. Conversely, an assignment with one true input extends to a satisfying assignment by activating only its two coordinates and extending the recursive AMO encodings. With all inputs false, all coordinate variables may be set false. Thus, projection onto the input variables yields exactly the AMO relation.

(a) Exactly One Queen Per Row. Let $R_i = (B_{i,0}, \dots, B_{i,n-1})$. The row AMO constraints are

$$F_{\text{row,prod}}^{\text{AMO}} = \bigwedge_{i=0}^{n-1} E_{\text{prod}}(R_i). \tag{120}$$

For groups above the cutoff, each call introduces independent coordinate vectors and applies Equation (117).

To illustrate the coordinate construction explicitly, consider row 0 of a 4×4 board before applying the Pairwise cutoff. A 2×2 grid gives

$$\begin{aligned}
&(r_1 \to \neg r_2) \wedge (c_1 \to \neg c_2) \\
&\wedge (B_{0,0} \to r_1) \wedge (B_{0,0} \to c_1) \\
&\wedge (B_{0,1} \to r_1) \wedge (B_{0,1} \to c_2) \\
&\wedge (B_{0,2} \to r_2) \wedge (B_{0,2} \to c_1) \\
&\wedge (B_{0,3} \to r_2) \wedge (B_{0,3} \to c_2).
\end{aligned} \tag{121}$$

Its full ten-clause expansion is

$$\begin{aligned}
\widetilde{F}_{\text{row},0}^{\text{AMO}} = &(\neg r_1 \vee \neg r_2) \\
&\wedge (\neg c_1 \vee \neg c_2) \\
&\wedge (\neg B_{0,0} \vee r_1) \\
&\wedge (\neg B_{0,0} \vee c_1) \\
&\wedge (\neg B_{0,1} \vee r_1) \\
&\wedge (\neg B_{0,1} \vee c_2) \\
&\wedge (\neg B_{0,2} \vee r_2) \\
&\wedge (\neg B_{0,2} \vee c_1) \\
&\wedge (\neg B_{0,3} \vee r_2) \\
&\wedge (\neg B_{0,3} \vee c_2).
\end{aligned} \tag{122}$$

The tilde identifies this illustrative grid expansion. The implemented encoding instead applies the $m \leq 4$ base case and emits the following six clauses:

$$\begin{aligned}
F_{\text{row},0,\text{prod}}^{\text{AMO}} = &(\neg B_{0,0} \vee \neg B_{0,1}) \\
&\wedge (\neg B_{0,0} \vee \neg B_{0,2}) \\
&\wedge (\neg B_{0,0} \vee \neg B_{0,3}) \\
&\wedge (\neg B_{0,1} \vee \neg B_{0,2}) \\
&\wedge (\neg B_{0,1} \vee \neg B_{0,3}) \\
&\wedge (\neg B_{0,2} \vee \neg B_{0,3}).
\end{aligned} \tag{123}$$

At least one queen per row. The row ALO formula is

$$F_{\text{row}}^{\text{ALO}} = \bigwedge_{i=0}^{n-1} \left(\bigvee_{j=0}^{n-1} B_{i,j} \right). \tag{124}$$

For $n = 4$, it expands into

$$\begin{aligned} F_{\text{row}}^{\text{ALO}} = {} & (B_{0,0} \vee B_{0,1} \vee B_{0,2} \vee B_{0,3}) \\ & \wedge (B_{1,0} \vee B_{1,1} \vee B_{1,2} \vee B_{1,3}) \\ & \wedge (B_{2,0} \vee B_{2,1} \vee B_{2,2} \vee B_{2,3}) \\ & \wedge (B_{3,0} \vee B_{3,1} \vee B_{3,2} \vee B_{3,3}). \end{aligned} \tag{125}$$

(b) Exactly One Queen Per Column. Let $C_j = (B_{0,j}, \dots, B_{n-1,j})$. The column AMO formula is

$$F_{\text{col,prod}}^{\text{AMO}} = \bigwedge_{j=0}^{n-1} E_{\text{prod}}(C_j). \tag{126}$$

For $n = 4$, column 0 has four inputs, so the implemented base case produces

$$\begin{aligned} F_{\text{col,0,prod}}^{\text{AMO}} = {} & (\neg B_{0,0} \vee \neg B_{1,0}) \\ & \wedge (\neg B_{0,0} \vee \neg B_{2,0}) \\ & \wedge (\neg B_{0,0} \vee \neg B_{3,0}) \\ & \wedge (\neg B_{1,0} \vee \neg B_{2,0}) \\ & \wedge (\neg B_{1,0} \vee \neg B_{3,0}) \\ & \wedge (\neg B_{2,0} \vee \neg B_{3,0}). \end{aligned} \tag{127}$$

A conceptual grid expansion would map these inputs to $(1,1), (1,2), (2,1), (2,2)$, respectively, but no coordinate variables are allocated at this size.

At least one queen per column. The column ALO formula is

$$F_{\text{col}}^{\text{ALO}} = \bigwedge_{j=0}^{n-1} \left(\bigvee_{i=0}^{n-1} B_{i,j} \right). \tag{128}$$

For $n = 4$, its four clauses are

$$\begin{aligned} F_{\text{col}}^{\text{ALO}} = {} & (B_{0,0} \vee B_{1,0} \vee B_{2,0} \vee B_{3,0}) \\ & \wedge (B_{0,1} \vee B_{1,1} \vee B_{2,1} \vee B_{3,1}) \\ & \wedge (B_{0,2} \vee B_{1,2} \vee B_{2,2} \vee B_{3,2}) \\ & \wedge (B_{0,3} \vee B_{1,3} \vee B_{2,3} \vee B_{3,3}). \end{aligned} \tag{129}$$

(c) At Most One Queen Per Diagonal. Diagonal inputs are ordered by increasing row index. Each diagonal is encoded independently, without an ALO constraint.

Main diagonals and their parallels. For $d = i - j$, define

$$\begin{aligned} a_d &= \max(0, d), \\ m_d &= n - |d|, \\ D_d &= \left(B_{a_d+t,\, a_d+t-d} \right)_{t=0}^{m_d-1}. \end{aligned} \tag{130}$$

The main-diagonal formula is

$$F_{\text{main,prod}}^{\text{AMO}} = \bigwedge_{d=2-n}^{n-2} E_{\text{prod}}(D_d). \tag{131}$$

For $n = 4$ and $d = 0$, the inputs are $(B_{0,0}, B_{1,1}, B_{2,2}, B_{3,3})$. Although their conceptual Product grid is 2×2, the implemented base case emits

$$\begin{aligned} F^{\mathrm{AMO}}_{\mathrm{main},0,\mathrm{prod}} = {} & (\neg B_{0,0} \vee \neg B_{1,1}) \\ & \wedge (\neg B_{0,0} \vee \neg B_{2,2}) \\ & \wedge (\neg B_{0,0} \vee \neg B_{3,3}) \\ & \wedge (\neg B_{1,1} \vee \neg B_{2,2}) \\ & \wedge (\neg B_{1,1} \vee \neg B_{3,3}) \\ & \wedge (\neg B_{2,2} \vee \neg B_{3,3}). \end{aligned} \tag{132}$$

Anti-diagonals and their parallels. For $s = i + j$, define

$$\begin{aligned} a_s &= \max(0, s - n + 1), \\ m_s &= \min(n - 1, s) - a_s + 1, \\ A_s &= \left(B_{a_s+t,\, s-a_s-t}\right)_{t=0}^{m_s-1}. \end{aligned} \tag{133}$$

The anti-diagonal formula is

$$F^{\mathrm{AMO}}_{\mathrm{anti},\mathrm{prod}} = \bigwedge_{s=1}^{2n-3} E_{\mathrm{prod}}(A_s). \tag{134}$$

For $n = 4$ and $s = 2$, the inputs are $(B_{0,2}, B_{1,1}, B_{2,0})$. A conceptual 2×2 mapping would leave position $(2, 2)$ unused. The actual implementation instead uses the three-clause Pairwise base case:

$$\begin{aligned} F^{\mathrm{AMO}}_{\mathrm{anti},2,\mathrm{prod}} = {} & (\neg B_{0,2} \vee \neg B_{1,1}) \\ & \wedge (\neg B_{0,2} \vee \neg B_{2,0}) \\ & \wedge (\neg B_{1,1} \vee \neg B_{2,0}). \end{aligned} \tag{135}$$

3.6.3. Complexity Analysis

Let $a_{\mathrm{prod}}(m)$ and $c_{\mathrm{prod}}(m)$ denote the auxiliary-variable and clause counts of one AMO constraint. The exact base cases are

$$a_{\mathrm{prod}}(m) = 0, \qquad c_{\mathrm{prod}}(m) = \binom{m}{2}, \qquad 0 \le m \le 4. \tag{136}$$

For $m > 4$, the coordinate variables and forward links give the recurrences

$$\begin{aligned} a_{\mathrm{prod}}(m) &= p + q + a_{\mathrm{prod}}(p) + a_{\mathrm{prod}}(q), \\ c_{\mathrm{prod}}(m) &= 2m + c_{\mathrm{prod}}(p) + c_{\mathrm{prod}}(q), \end{aligned} \tag{137}$$

where $p = \lceil \sqrt{m} \rceil$ and $q = \lceil m/p \rceil$. The term $p + q$ counts only the current level's auxiliary variables; further variables arise recursively.

Since $p, q = \Theta(\sqrt{m})$, these recurrences yield

$$\begin{aligned} a_{\mathrm{prod}}(m) &= \Theta(\sqrt{m}), \\ c_{\mathrm{prod}}(m) &= 2m + O(\sqrt{m}) = \Theta(m). \end{aligned} \tag{138}$$

The expression $2m + \binom{p}{2} + \binom{q}{2}$ instead describes a construction that applies Pairwise directly to both coordinate sets. It coincides with the recursive construction when both coordinate sets fall within the Pairwise base case.

For the complete board with $n \geq 2$, the exact counts are

$$\begin{aligned} A^{\text{board}}_{\text{prod}}(n) &= (2n+2)a_{\text{prod}}(n) \\ &\quad + 4\sum_{m=2}^{n-1} a_{\text{prod}}(m), \\ V_{\text{prod}}(n) &= n^2 + A^{\text{board}}_{\text{prod}}(n), \\ C_{\text{prod}}(n) &= 2n + (2n+2)c_{\text{prod}}(n) \\ &\quad + 4\sum_{m=2}^{n-1} c_{\text{prod}}(m). \end{aligned} \tag{139}$$

Consequently,

$$\begin{aligned} A^{\text{board}}_{\text{prod}}(n) &= \Theta(n^{3/2}), \\ V_{\text{prod}}(n) &= n^2 + \Theta(n^{3/2}) = \Theta(n^2), \\ C_{\text{prod}}(n) &= \Theta(n^2). \end{aligned} \tag{140}$$

At $n = 4$, every group falls within the Pairwise base case. The complete formula therefore contains 16 board variables, zero auxiliary variables, and 84 clauses. At $n = 1331$, the recursive construction contains 765,004 auxiliary variables and 2,536,565 total variables.

Product's auxiliary-variable count grows more slowly than those of Sequential Counter and Commander. Binary, however, has the smaller asymptotic auxiliary count $\Theta(n \log n)$. Product achieves quadratic clause growth, as do Sequential Counter and Commander; this asymptotic classification does not establish a globally minimum clause count across all AMO encodings.

3.7. Theoretical Complexity Summary Table and Independent Verification

3.7.1. Theoretical Complexity Summary

Let $a_E(m)$ and $c_E(m)$ denote the auxiliary-variable and clause counts for a single AMO constraint on m inputs under encoding E. Define the aggregation operator

$$\mathcal{T}_n[f] = (2n+2)f(n) + 4\sum_{m=2}^{n-1} f(m), \qquad n \geq 2. \tag{141}$$

The first term accounts for the $2n$ rows and columns and the two central diagonals. The second term accounts for four diagonals of every length from 2 to $n-1$.

For any of the five encodings, the complete board formula therefore has

$$\begin{aligned} V_E(n) &= n^2 + \mathcal{T}_n[a_E], \\ C_E(n) &= 2n + \mathcal{T}_n[c_E]. \end{aligned} \tag{142}$$

The $2n$ additional clauses are the row and column ALO clauses. Table 1 summarizes the exact counts or recurrences and their asymptotic growth.

For a single group, Commander has $a_{\text{com}}(m) = m/2 + O(\log m)$ and $c_{\text{com}}(m) = 7m/2 + O(\log m)$. Product has $a_{\text{prod}}(m) = \Theta(\sqrt{m})$ and $c_{\text{prod}}(m) = 2m + O(\sqrt{m})$. All encodings generate zero AMO clauses and zero auxiliary variables for groups of length at most one.

The distinction between auxiliary variables and total variables is essential. Binary introduces $\Theta(n \log n)$ auxiliaries, while Product introduces $\Theta(n^{3/2})$ auxiliaries. Both complete formulas still contain $\Theta(n^2)$ variables because the n^2 board variables are present in every encoding. Sequential Counter and Commander each introduce $\Theta(n^2)$ auxiliary variables.

At $n = 1$, all five implementations produce one board variable and two identical positive unit clauses, corresponding to the separately retained row and column ALO constraints.

Table 1
Theoretical Complexity Comparison of Five SAT Encodings for the N-Queens Problem

Encoding Scheme	Auxiliary Variables per Line	Clauses per Line	Total Formula Variables $V_E(n)$	Total Formula Clauses $C_E(n)$	Asymptotic Clause Complexity
Binomial	0	$\binom{m}{2}$	n^2	$\frac{5n^3 - 6n^2 + 7n}{3}$	$\Theta(n^3)$
Binary	$\ell(m)$	$m\ell(m)$	$n^2 + \mathcal{T}_n[\ell]$	$2n + \mathcal{T}_n[m \mapsto m\ell(m)]$	$\Theta(n^2 \log n)$
Sequential Counter	$m - 1$	$3m - 4$	$5n^2 - 6n + 2$	$12n^2 - 22n + 12$	$\Theta(n^2)$
Commander	$a_{\text{com}}(m)$	$c_{\text{com}}(m)$	$n^2 + \mathcal{T}_n[a_{\text{com}}]$	$2n + \mathcal{T}_n[c_{\text{com}}]$	$\Theta(n^2)$
Recursive 2-Product	$a_{\text{prod}}(m)$	$c_{\text{prod}}(m)$	$n^2 + \mathcal{T}_n[a_{\text{prod}}]$	$2n + \mathcal{T}_n[c_{\text{prod}}]$	$\Theta(n^2)$

Here $\ell(m) = \lceil \log_2 m \rceil$ for $m \geq 2$. Per-line entries describe AMO clauses only; total clause counts include ALO. All displayed board formulas apply for $n \geq 2$. Commander uses the exact recurrences in Equation (105), with Pairwise at $m \leq 3$. Product uses Equation (137), with Pairwise at $m \leq 4$.

3.7.2. Independent Verification and Solution Counting

Correctness was checked at both the individual AMO level and the complete board level. The verification distinguishes assignments to board variables from assignments to auxiliary variables, so alternative auxiliary assignments do not create additional counted placements.

Exhaustive AMO consistency checks. For each encoding and every group length $m \in \{0, \dots, 10\}$, all 2^m input assignments were enumerated. Each assignment was fixed through SAT assumptions, while auxiliary variables remained free. The resulting formula was satisfiable exactly when the input assignment contained at most one true variable:

$$\exists Y \; E(x, Y) \quad \Longleftrightarrow \quad \sum_{t=0}^{m-1} x_t \leq 1. \tag{143}$$

Across the five encodings, this produced

$$5 \sum_{m=0}^{10} 2^m = 5(2^{11} - 1) = 10{,}235 \tag{144}$$

input-assignment checks. All checks passed.

Unit-propagation checks. Propagation was evaluated on group lengths

$$m \in \{2, 3, 4, 5, 9, 10, 17, 64\}. \tag{145}$$

For every input position in every group, that input was set true and unit propagation was invoked. The check required a consistent propagation result containing the negation of every other input. The total number of cases was

$$5(2 + 3 + 4 + 5 + 9 + 10 + 17 + 64) = 570. \tag{146}$$

All five encodings passed every case.

Projected solution enumeration. For each encoding and every $n \in \{1, \dots, 8\}$, the complete N-Queens formula was solved repeatedly. Each satisfying assignment was projected onto the board variables and converted into a placement vector $Q = (q_0, \dots, q_{n-1})$.

After recording a placement, the following blocking clause was added:

$$\bigvee_{i=0}^{n-1} \neg B_{i,q_i}. \tag{147}$$

Because the board formula enforces exactly one queen per row, this clause excludes precisely that placement, including all of its possible auxiliary extensions. Enumeration continued until the solver returned UNSAT.

Independent permutation checker. A separate checker enumerated permutations of $\{0, \dots, n-1\}$. Each permutation already satisfies the row and column conditions. It was accepted exactly when

$$\begin{aligned} \big|\{q_i + i : 0 \le i < n\}\big| &= n, \\ \big|\{q_i - i : 0 \le i < n\}\big| &= n. \end{aligned} \tag{148}$$

The complete placement sets produced by SAT enumeration were compared with those accepted by the permutation checker, rather than comparing their cardinalities alone.

For all five encodings, both methods produced identical placement sets and the following counts:

n	1	2	3	4	5	6	7	8
Placements	1	0	0	2	10	4	40	92

(149)

These counts include all distinct board placements; rotations and reflections are not identified as a single solution. The checks jointly cover local AMO semantics, propagation from a selected input, and the composition of row, column, and diagonal constraints into the complete N-Queens formula.

4. Experiments and Results

4.1. Experimental setup and measurement scope

The experiments were conducted on a Lenovo computer equipped with an AMD Ryzen 7 6800H processor, 16 logical CPUs, and 16 GB of RAM, running 64-bit Windows 11 Home (build 26200). The Python implementation was evaluated on the following board sizes:

$$\begin{aligned} n \in \{&4, 8, 9, 16, 25, 27, 36, 49, 64, 81, 100, 125, \\ &216, 343, 512, 729, 1000, 1331\}. \end{aligned}$$

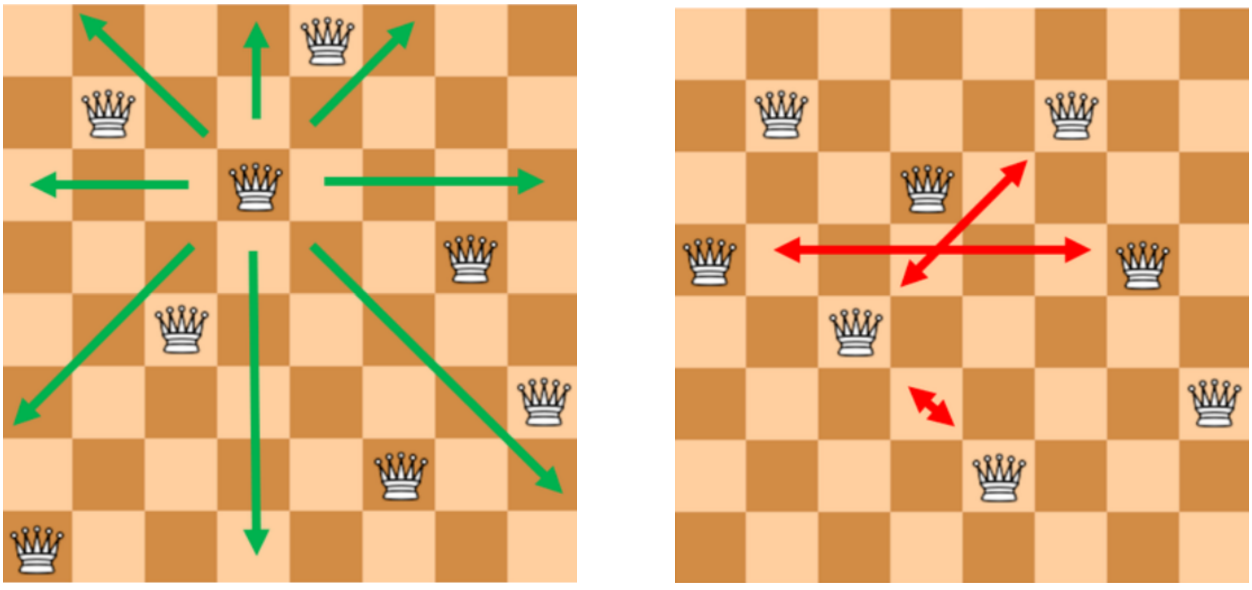
This set is the union of squares k^2 for $2 \le k \le 10$ and cubes k^3 for $2 \le k \le 11$, spanning approximately five orders of magnitude in the number of board cells.

Five AMO encodings are paired with `cadical195` and `glucose3` through PySAT [6]. Four additional configurations use `cp-sat`, `cplex-cp`, `gurobi`, and `cplex-mip`. The resulting benchmark comprises 14 configurations across 18 sizes, yielding 252 runs. Each configuration–size pair is executed once, so the median fields in `summary.csv` contain the corresponding observed measurements. CP and ILP configurations use one thread and seed zero; SAT configurations retain the backend defaults recorded in the CSV.

Each run executes in a separate spawned process under external supervision. The parent enforces a 100-second process budget and monitors process-tree RSS approximately every 20 ms against a threshold of 14,336 MiB (14 GiB). The field `Peak_RAM_MB` stores bytes divided by 2^{20} and is therefore reported in MiB. This measurement captures the resident footprint of the worker, imported libraries, model data, solver state, and observed child processes. The benchmark produced 197 successful runs and 55 external timeouts, with no recorded memory-limit termination.

Timing is separated into `Build_Time`, `Solve_Time`, and their sum, `Total_Time`. For SAT, construction includes Python CNF generation and solver bootstrap. For CP and ILP, it covers model construction before the timed solve call. `Solve_Time` measures the elapsed solver API call, including the initialization, serialization, presolve, propagation, and search performed within that interval. The external process budget and `Wall_Time` additionally cover process startup, imports, solution extraction, and worker cleanup. Parent-side validation and CSV output follow this interval. This separation enables analysis of model construction, solver execution, and process lifecycle costs.

Tables round directly from the full CSV values: four decimal places below 0.01 s, three below 0.1 s, and two otherwise. Bold identifies the smallest unrounded successful measurement within each comparison. Timeouts are reported as censored outcomes rather than assigned a finite successful runtime. Every returned placement was independently checked for integer bounds, one queen per row, distinct columns, and distinct diagonal indices; all 197 stored solutions passed.



(a) Valid placement with green attack vectors.

(b) Invalid placement with red conflict vectors.

Figure 1: Illustration of (a) a valid non-attacking configuration, with attack directions indicated by green arrows, and (b) an invalid placement containing mutually attacking queens, highlighted by red arrows, for the N-Queens problem.

4.2. Experiment 1: Formula sizes

Table 2 reports exact formula sizes computed using Equation (**??**). All 168 SAT records containing model counts match these values. Figure 2 distinguishes calculated counts from sizes observed after model construction, including constructions followed by a solver timeout.

Table 2
Exact input formula sizes. V includes board and auxiliary variables; C includes the $2n$ ALO clauses. A dagger means neither backend recorded the size, so the value is calculated from the recurrence only.

n		Pairwise	Binary	Sequential	Commander	Product
4	V	16	48	58	36	16
	C	84	120	116	124	84
16	V	256	572	1 186	744	820
	C	6 352	3 928	2 732	3 176	2 804
64	V	4 096	6 136	20 098	12 442	12 476
	C	428 864	95 576	47 756	55 990	45 620
216	V	46 656	55 988	231 986	140 030	94 672
	C	16 703 352	1 449 224	555 132	647 438	458 024
512	V	262 144†	287 730	1 307 650	789 618	459 472
	C	223 173 120†	9 262 424	3 134 476	3 662 390	2 439 384
1331	V	1 771 561†	1 851 197†	8 849 821	5 326 901	2 536 565
	C	3 926 372 802†	75 151 050†	21 229 462	24 787 642	15 533 774

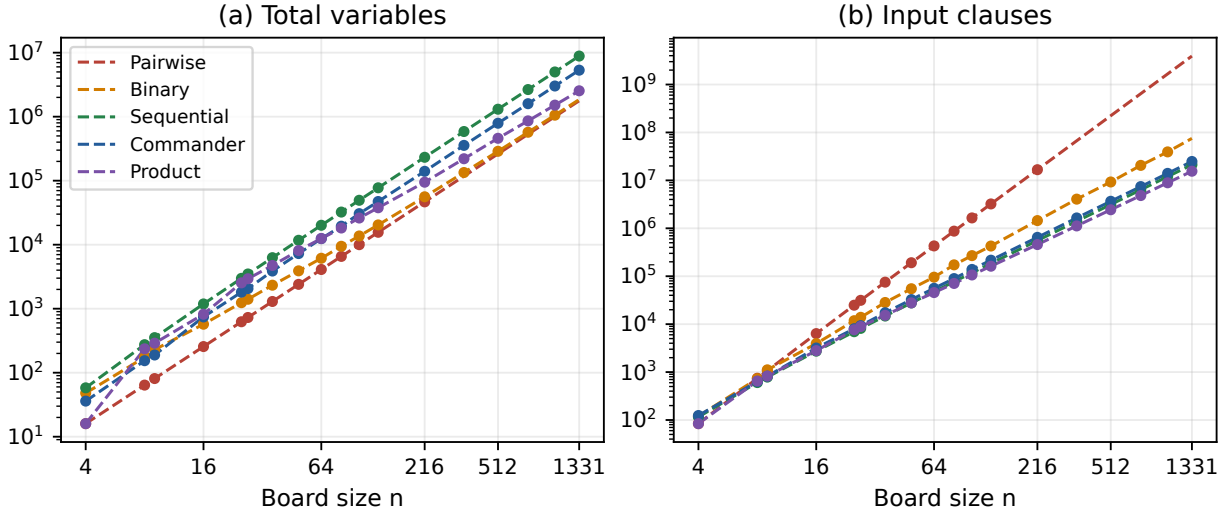


Figure 2: Exact formula counts across the tested sizes. Dashed lines show the count formulas; markers identify sizes recorded after model construction by at least one SAT backend, including timed-out solves.

The counts expose a pronounced trade-off between variable economy and clause growth. Pairwise uses only n^2 variables, yet generates 16,703,352 clauses at $n = 216$ and reaches a calculated 3,926,372,802 clauses at $n = 1331$. Binary reduces the latter count to 75,151,050 by introducing index bits. Eliminating auxiliary variables therefore carries a substantial cost in the number of explicit conflicts represented.

Sequential and Commander use quadratic auxiliary space, whereas Product uses subquadratic auxiliary space and achieves the smallest clause count among the five variants at every tested size $n \geq 64$. At $n = 1331$, Product contains 2,536,565 variables and 15,533,774 clauses. Its 765,004 auxiliary variables add 43.18% to the board-variable count, while Pairwise requires approximately 252.76 times as many clauses. These reductions establish Product's compactness

and set up the central empirical question: how effectively does each backend translate a smaller formula into faster solving?

4.3. Experiment 2: SAT runtime

Table 3 compares all ten SAT configurations using recorded build-plus-solver-call time. Winners are selected from unrounded measurements, preserving distinctions between short runs. Figure 3 displays successful measurements and leaves gaps at timeouts.

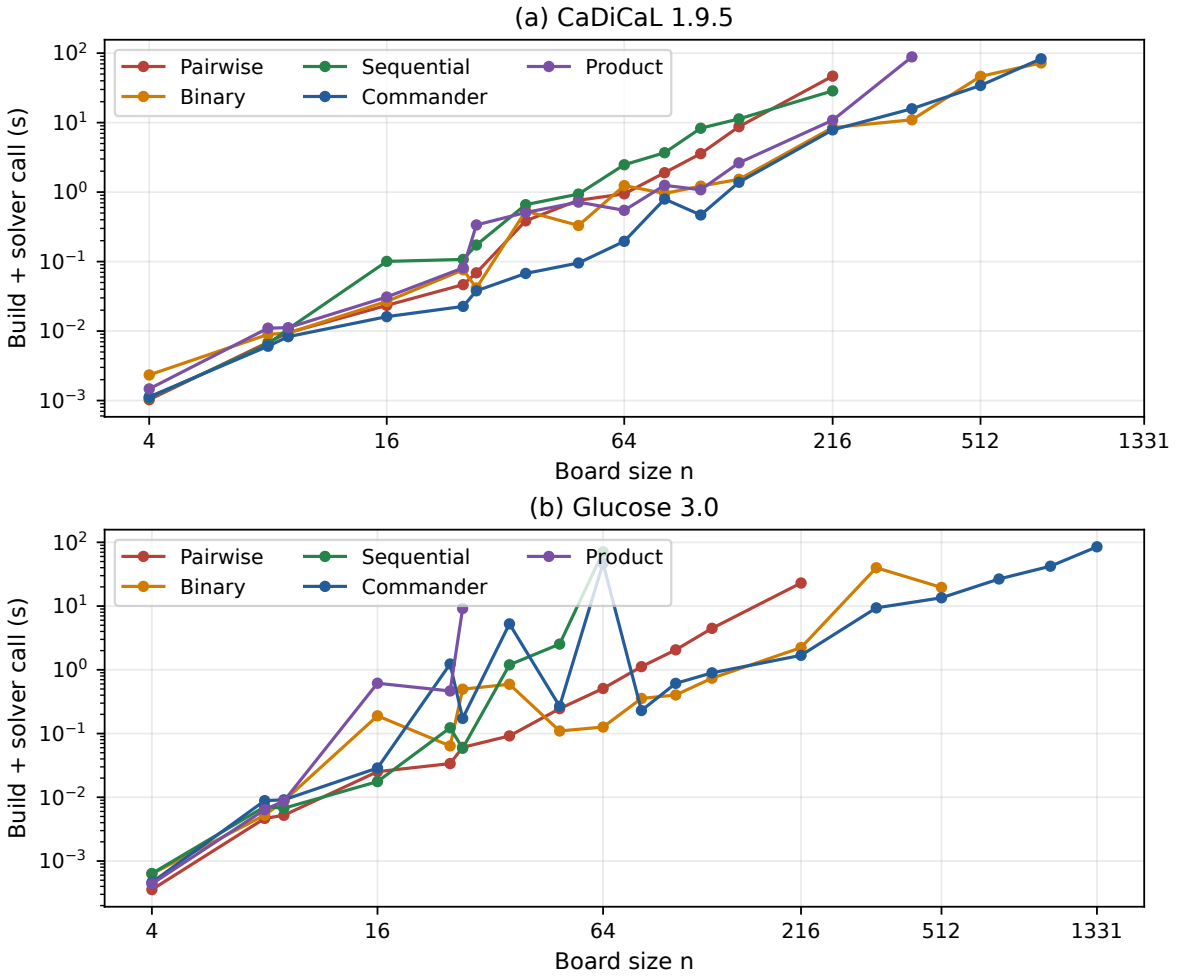


Figure 3: Build-plus-solver-call time for the five AMO encodings. Only successful measurements are connected across adjacent tested sizes.

Commander with Glucose achieves the strongest SAT coverage, solving all 18 sizes and reaching $n = 1000$ and $n = 1331$ in 42.21 and 85.08 s, respectively. Binary and Commander with CaDiCaL each solve 16 sizes, with their runtime ranking changing across instances. At $n = 343$, Binary takes 10.98 s against Commander's 15.83 s; at $n = 512$, Commander leads with 34.23 s against Binary's 46.29 s. Backend and encoding must therefore be evaluated jointly.

The results reveal a clear size–speed paradox. Product provides the smallest clause count at the larger tested sizes, yet Product with Glucose solves only 6 sizes and times out at every tested $n \geq 36$. Commander uses more clauses but, with the same backend, solves every size. Switching Product to CaDiCaL raises its coverage from 6 to 14 sizes. Sequential exhibits the same backend sensitivity, solving 9 sizes with Glucose and 13 with CaDiCaL. Formula compactness is thus a distinct property from computational efficiency.

Table 3
Recorded SAT Total_Time in seconds. Bold indicates the fastest encoding separately for each backend and size; the final row counts validated solutions. TO denotes an external timeout.

CaDiCaL 1.9.5					
n	Pairwise	Binary	Sequential	Commander	Product
4	**0.0010**	0.0023	0.0011	0.0011	0.0015
8	0.0069	0.0090	0.0064	**0.0060**	0.011
9	0.0095	0.0094	0.011	**0.0083**	0.011
16	0.023	0.027	0.10	**0.016**	0.031
25	0.047	0.076	0.081	**0.023**	0.081
27	0.069	0.042	0.17	**0.038**	0.34
36	0.39	0.53	0.66	**0.068**	0.51
49	0.76	0.33	0.94	**0.096**	0.72
64	0.94	1.24	2.48	**0.20**	0.55
81	1.90	0.96	3.69	**0.79**	1.25
100	3.58	1.22	8.30	**0.47**	1.08
125	8.74	1.53	11.32	**1.39**	2.64
216	46.66	8.48	28.64	**7.86**	10.83
343	TO	**10.98**	TO	15.83	88.28
512	TO	46.29	TO	**34.23**	TO
729	TO	**72.21**	TO	82.95	TO
1000	TO	TO	TO	TO	TO
1331	TO	TO	TO	TO	TO
Solved/18	13	**16**	13	**16**	14

Glucose 3.0					
n	Pairwise	Binary	Sequential	Commander	Product
4	**0.0004**	0.0006	0.0006	0.0005	0.0004
8	**0.0047**	0.0052	0.0071	0.0089	0.0064
9	**0.0052**	0.0088	0.0067	0.0091	0.0087
16	0.025	0.19	**0.018**	0.029	0.62
25	**0.034**	0.064	0.12	1.23	0.47
27	0.061	0.50	**0.059**	0.17	9.16
36	**0.092**	0.59	1.20	5.26	TO
49	0.25	**0.11**	2.53	0.27	TO
64	0.51	**0.13**	71.17	47.01	TO
81	1.12	0.36	TO	**0.23**	TO
100	2.06	**0.40**	TO	0.61	TO
125	4.48	**0.74**	TO	0.90	TO
216	23.01	2.23	TO	**1.69**	TO
343	TO	39.93	TO	**9.36**	TO
512	TO	19.61	TO	**13.49**	TO
729	TO	TO	TO	**26.60**	TO
1000	TO	TO	TO	**42.21**	TO
1331	TO	TO	TO	**85.08**	TO
Solved/18	13	15	9	**18**	6

Runtime also scales non-monotonically. Commander with Glucose takes 0.2678, 47.0071, and 0.2298 s at $n = 49, 64, 81$, respectively: a sharp intermediate spike followed by a faster solution on the larger board. Product with CaDiCaL similarly decreases from 0.7170 s at $n = 49$ to 0.5466 s at $n = 64$. These reversals show that increasing board size does not induce a corresponding ordering of computational difficulty for the evaluated solvers.

The encodings present different propagation structures to the backend: Binary links inputs directly to index bits, Sequential organizes implications along a linear chain, and Commander builds a hierarchy with $O(\log k)$ levels for a line of length k. The observed runtime and coverage differences establish the practical importance of these representation choices in combination with the solver. Clause count remains a useful measure of representation cost, but successful configuration selection requires measured solving performance.

4.4. Experiment 3: SAT, CP, and ILP comparison

Table 4 compares four representative SAT configurations with the four CP and ILP baselines. Figure 4 complements the total-time comparison by separating out the solver-call interval.

Table 4
Recorded Total_Time in seconds. CG = Commander+Glucose, CC = Commander+CaDiCaL, BC = Binary+CaDiCaL, PC = Product+CaDiCaL. Bold identifies the fastest displayed configuration using unrounded measurements.

n	CG	CC	BC	PC	CP-SAT	CPLEX CP	Gurobi	CPLEX MIP
4	**0.0005**	0.0011	0.0023	0.0015	0.0073	0.052	0.0055	0.031
8	0.0089	0.0060	0.0090	0.011	0.017	0.042	**0.0054**	0.033
9	0.0091	0.0083	0.0094	0.011	0.020	0.037	**0.0062**	0.034
16	0.029	**0.016**	0.027	0.031	0.035	0.050	0.020	0.034
25	1.23	0.023	0.076	0.081	0.080	0.049	**0.015**	0.059
27	0.17	**0.038**	0.042	0.34	0.087	0.043	0.041	0.053
36	5.26	0.068	0.53	0.51	0.15	**0.051**	0.064	0.090
49	0.27	0.096	0.33	0.72	0.28	0.049	**0.026**	0.12
64	47.01	0.20	1.24	0.55	0.61	0.057	**0.056**	0.18
81	0.23	0.79	0.96	1.25	1.05	0.071	**0.064**	0.28
100	0.61	0.47	1.22	1.08	1.63	**0.049**	0.076	0.41
125	0.90	1.39	1.53	2.64	3.32	**0.064**	0.16	0.70
216	1.69	7.86	8.48	10.83	19.80	**0.071**	0.38	1.84
343	9.36	15.83	10.98	88.28	T0	**0.68**	0.99	7.34
512	13.49	34.23	46.29	T0	T0	**0.65**	2.18	11.94
729	26.60	82.95	72.21	T0	T0	**0.73**	43.47	T0
1000	42.21	T0	T0	T0	T0	**0.36**	T0	42.74
1331	85.08	T0	T0	T0	T0	**0.40**	21.31	T0
Solved/18	**18**	16	16	14	13	**18**	17	16

CPLEX CP delivers the strongest large-instance performance. It solves all 18 sizes, keeps every recorded total below 0.73 s, and completes $n = 1331$ in 0.3971 s. Its maximum is 0.7273 s at $n = 729$, and it is the fastest displayed configuration at every tested size $n \geq 100$. In contrast, CP-SAT solves 13 sizes and requires 19.7989 s at $n = 216$. The gap between these two backends shows that a shared AllDifferent formulation can lead to substantially different computational performance.

The MIP baselines are also highly competitive. Gurobi solves 17 sizes and CPLEX MIP solves 16, exceeding the coverage of most SAT configurations and CP-SAT. At $n = 512$, Gurobi completes in 2.18 s and CPLEX MIP in 11.94 s, compared with 13.49 s for the fastest SAT configuration. The results place binary ILP formulations among the strongest approaches evaluated for this classical combinatorial feasibility problem.

Both MIP backends exhibit pronounced non-monotonic behavior. Gurobi times out at $n = 1000$ but solves $n = 1331$ in 21.3057 s, comprising 20.0195 s of construction and only 1.2862 s in the solver call. CPLEX MIP times out at $n = 729$ yet solves $n = 1000$ in 42.7350 s. Larger instances can therefore be substantially easier for a particular solver, making instance-by-instance coverage essential to the comparison.

Process lifecycle measurements add a second perspective to these short runtimes. Pairwise with Glucose at $n = 4$ records Total_Time = 0.0003563 s and Wall_Time = 1.5253978 s. At $n = 1331$, CPLEX CP records Total_Time = 0.3971 s and Wall_Time = 1.7099 s. Model-and-solver performance and process-level application latency are therefore separate quantities, especially when initialization costs exceed the timed computation.

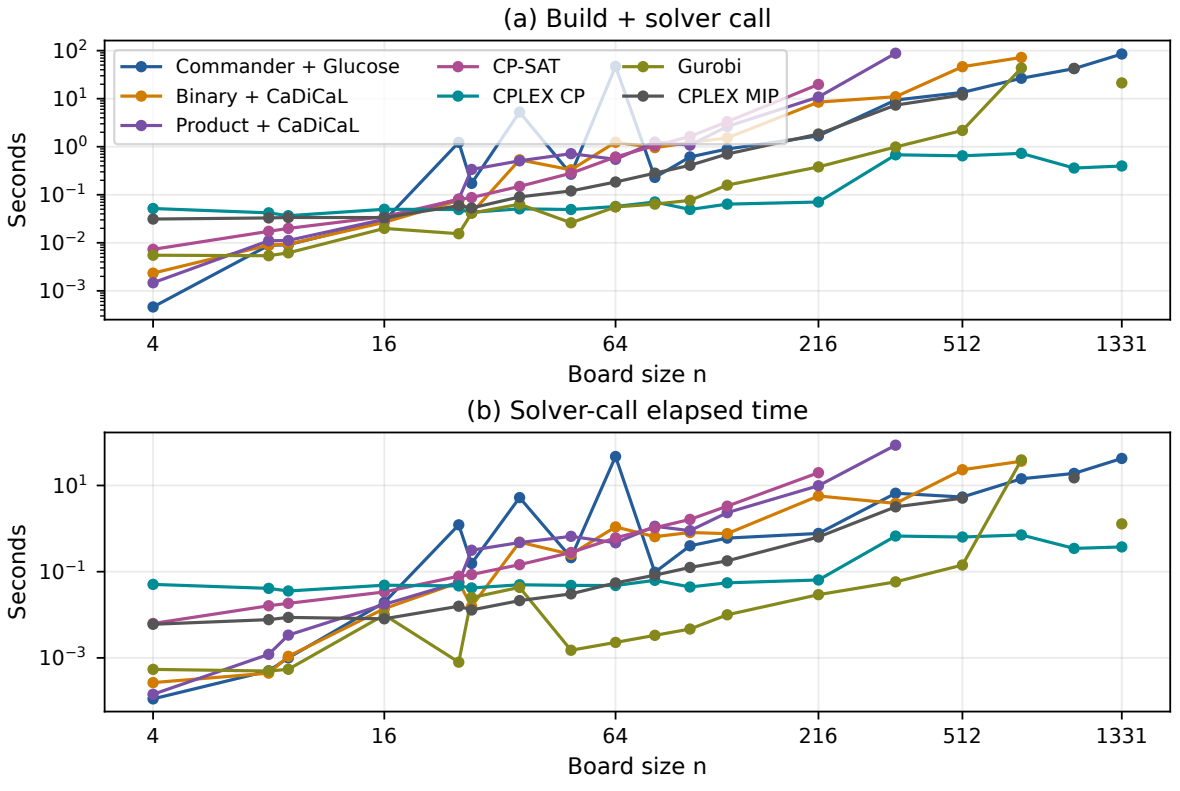


Figure 4: Representative successful runs: (a) reported Total_Time and (b) elapsed time around the solver API call. Missing points are not interpolated. The lower panel is not a measurement of pure internal search.

Taken together, the comparison establishes a configuration-specific performance hierarchy. CP Optimizer leads on the larger boards, Commander with Glucose provides complete SAT coverage, and the MIP baselines outperform most SAT configurations in solved-instance count. These distinctions are more informative than grouping all implementations under a single SAT, CP, or ILP label.

4.5. Experiment 4: Memory and construction overhead

Figure 5 examines two resource costs that are obscured by a single runtime total: resident memory and model construction. The memory panel contains successful runs for the five CaDiCaL encodings and four baselines; the timing panel compares their construction and solver-call intervals at $n = 216$.

Variable count alone is a poor indicator of resident memory. At $n = 216$, Pairwise with CaDiCaL uses 4,378.09 MiB, compared with 512.91 MiB for Binary, 363.05 MiB for Commander, 343.30 MiB for Sequential, and 275.25 MiB for Product. Pairwise eliminates auxiliary variables yet has by far the largest measured footprint among these configurations. At $n = 729$, Binary contains 20,558,002 input clauses and uses 5,738.11 MiB, whereas Commander contains 7,421,982 clauses and uses 2,736.89 MiB. Compact variable representations can therefore coexist with substantial process memory costs.

The cross-paradigm differences remain pronounced on large boards. At $n = 1331$, CPLEX CP uses 146.19 MiB, Gurobi uses 1,243.93 MiB, and Commander with Glucose uses 6,177.94 MiB. CPLEX MIP uses 1,697.57 MiB at $n = 1000$. CP Optimizer thus combines complete coverage and short runtimes with a small measured memory footprint at the largest size.

Table 5 exposes a substantial construction bottleneck. For Gurobi at $n = 216$, the solver call takes only 0.0293 s, while building the model consumes 92.30% of the total. CPLEX MIP also spends most of its time constructing the model, with a build share of 65.28%. For Pairwise with CaDiCaL, CNF generation and bootstrap consume 27.9064 s,

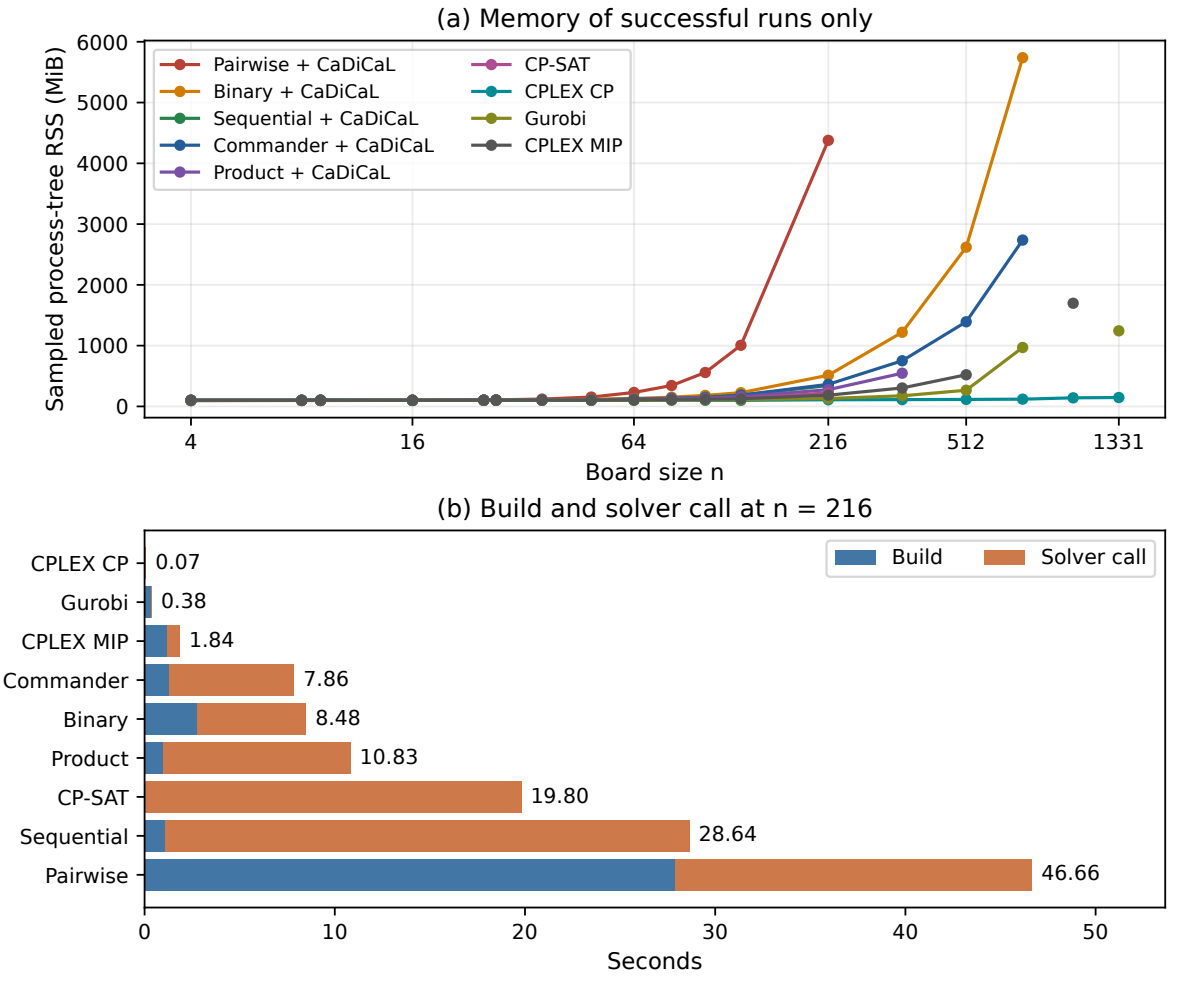


Figure 5: (a) Sampled RSS of successful runs, in MiB, for five CaDiCaL encodings and four baselines. (b) Measured build/solver-call split at $n = 216$, sorted by total. The bar-end values are totals in seconds.

Table 5
Timing decomposition at $n = 216$, in seconds. SAT rows use CaDiCaL. Percentages are computed from unrounded values, not from the displayed rounded numbers.

Method	Build	Solver call	Total	Build share
binomial	27.9064	18.7543	46.6607	59.81%
binary	2.7757	5.7075	8.4832	32.72%
sequential	1.0602	27.5786	28.6388	3.70%
commander	1.3090	6.5533	7.8622	16.65%
product	0.9552	9.8702	10.8254	8.82%
cp-sat	0.0136	19.7853	19.7989	0.07%
cplex-cp	0.0064	0.0642	0.0706	9.11%
gurobi	0.3510	0.0293	0.3803	92.30%
cplex-mip	1.1995	0.6380	1.8375	65.28%

accounting for 59.81% of the 46.6607 s total. Reporting only the solver-call interval would omit the dominant cost for these configurations.

The timing decomposition also explains why compact construction does not ensure a short total. Product builds in 0.9552 s but spends 9.8702 s in the solver call. CP-SAT constructs its model in 0.0136 s, while 99.93% of its total lies in the solver-call interval. Different configurations therefore expose different optimization targets: construction dominates some methods, whereas solver execution dominates others.

Construction remains substantial at the largest size. Commander with Glucose spends 42.7036 s building and 42.3752 s in the solver call at $n = 1331$, dividing its total almost equally between the two stages. Commander with CaDiCaL completes construction in 57.9102 s before the run reaches the external timeout. Under a fixed process budget, model construction directly reduces the time available for subsequent solver execution.

4.6. Experiment 5: Coverage and comparative synthesis

Figure 6 combines speed and coverage by sorting each configuration's successful Total_Time values. The vertical coordinate gives the runtime of each ranked instance, while the endpoint records the number of validated solutions. Figure 7 complements this view with the full configuration-by-size matrix, making runtime spikes and timeout patterns explicit.

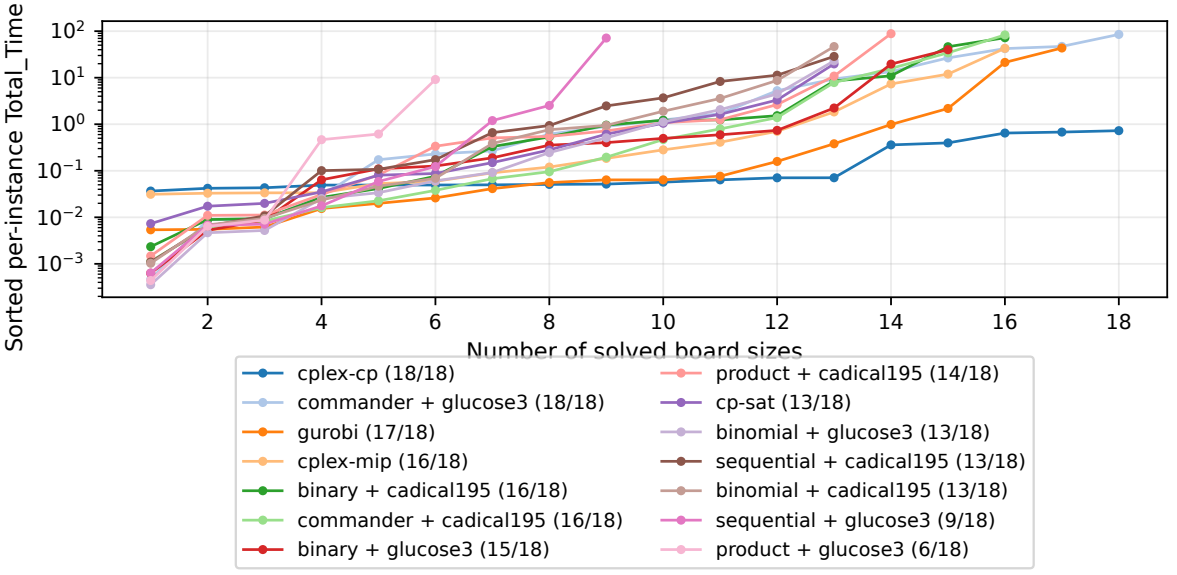


Figure 6: Cactus plot of the 14 configurations. Curves stop at the number of validated solutions; no finite successful runtime is assigned to a timeout.

CPLEX CP and Commander+Glucose achieve complete coverage at 18/18. Gurobi follows with 17/18, while CPLEX MIP, Binary+CaDiCaL, and Commander+CaDiCaL each achieve 16/18. Binary+Glucose solves 15 sizes and Product+CaDiCaL solves 14. CP-SAT, both Pairwise configurations, and Sequential+CaDiCaL each solve 13 sizes; Sequential+Glucose solves 9 and Product+Glucose solves 6.

The combined results identify three practical determinants of performance. First, encoding and backend interact strongly: changing Product's backend increases coverage from 6 to 14 sizes, while Commander with Glucose reaches all 18. Second, smaller formulas can yield slower computation, so clause minimization and runtime optimization require separate evaluation. Third, construction is a first-order component of total cost, reaching 92.30% for Gurobi at $n = 216$ and approximately half of Commander's successful Glucose run at $n = 1331$.

Coverage and timing provide complementary evidence. Non-monotonic outcomes make the largest solved size an incomplete summary, while the cactus plot distinguishes configurations that solve the same number of instances at substantially different speeds. Together with the memory and construction measurements, these views provide a quantitative basis for choosing an encoding–solver combination.

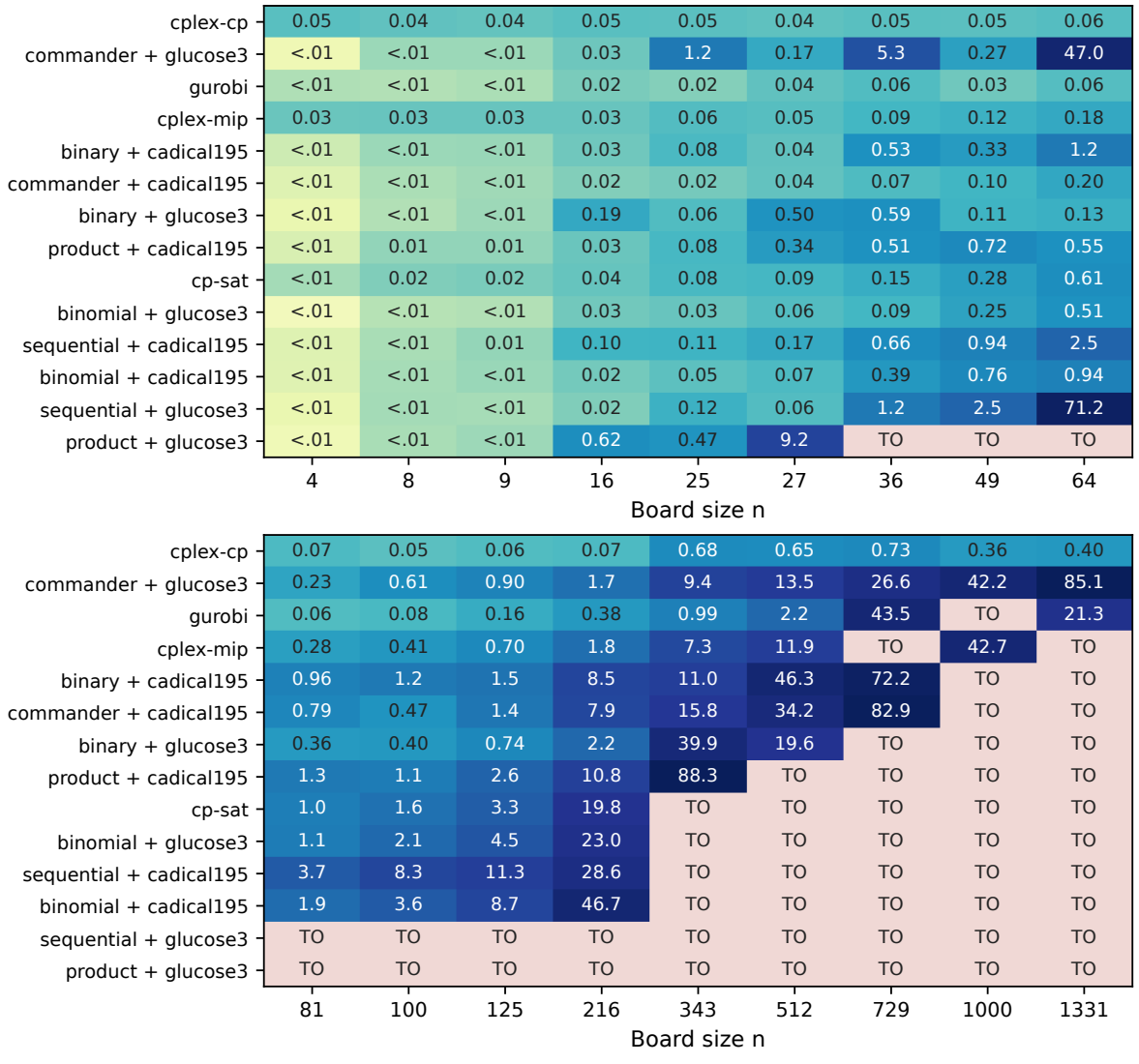


Figure 7: The complete 14-by-18 runtime matrix, split into two size ranges for readability. Values are rounded for display; values below 0.01 s are labeled < .01. Colors represent the logarithm of the unrounded successful total, with pink denoting Timeout. Table winners use the original unrounded values.

5. Conclusion

This study presents a quantitative comparison of five AMO encodings, two CDCL SAT backends, and four CP and ILP baselines for ordinary N-Queens feasibility. The 252 benchmark runs cover 18 board sizes up to $n = 1331$, with performance characterized through formula size, construction time, solver-call time, process memory, and validated solution coverage.

The comparison separates representational efficiency from computational efficiency. Pairwise has cubic whole-board clause growth, Binary has $\Theta(n^2 \log n)$ growth, and Sequential, Commander, and recursive Product have quadratic growth. Product achieves the smallest clause counts at the larger tested sizes, yet Commander with Glucose is the only SAT configuration to solve all 18 sizes. The resulting size–speed paradox makes encoding–backend compatibility central to practical SAT performance.

Across paradigms, CP Optimizer delivers the strongest large-board performance, solving every size in under 0.73 s and reaching $n = 1331$ in 0.3971 s. Gurobi and CPLEX MIP solve 17 and 16 sizes, respectively, outperforming most SAT configurations in coverage. Their non-monotonic runtimes further demonstrate that board size alone does not rank computational difficulty.

Finally, the timing decomposition establishes model construction as a major performance bottleneck. Construction accounts for 92.30% of Gurobi's total at $n = 216$ and approximately half of Commander+Glucose's total at $n = 1331$. Effective optimization therefore requires attention to the full model-and-solver pipeline. The study provides concrete guidance for this choice: evaluate formula compactness, solver compatibility, construction overhead, and memory consumption together. Future experiments can extend this analysis through repeated seeds, solver instrumentation, and controlled comparisons of propagation structures.

Data Availability

Project repository: `https://github.com/danh-nguyen2210/n_queens`. The report is accompanied by the original `runs.csv` and `summary.csv` measurements, source code, plotting script, and validation script. Reproduction uses the documented software environment and the applicable commercial solver licenses.

References

[1] I. P. Gent, C. Jefferson, P. Nightingale, Complexity of n-Queens Completion (Extended Abstract), Proceedings of IJCAI-18 (2018) 5608–5611. https://www.ijcai.org/proceedings/2018/0794.pdf.

[2] Google OR-Tools, The N-queens Problem, official documentation. https://developers.google.com/optimization/cp/queens.

[3] C. Sinz, Towards an Optimal CNF Encoding of Boolean Cardinality Constraints, Principles and Practice of Constraint Programming (2005). https://www.carstensinz.de/papers/CP-2005.pdf.

[4] W. Klieber, G. Kwon, Efficient CNF Encoding for Selecting 1 from N Objects (2007). https://www.cs.cmu.edu/~wklieber/papers/2007_efficient-cnf-encoding-for-selecting-1.pdf.

[5] J. Chen, A New SAT Encoding of the At-Most-One Constraint, Tenth International Workshop on Constraint Modelling and Reformulation (2010).

[6] PySAT developers, SAT solvers' API (`pysat.solvers`), official documentation. https://pysathq.github.io/docs/html/api/solvers.html.